TransKV: A Data-Driven Pruning Method for Large Foundation Models

Guangning Xu
Hong Kong Baptist University
xuguangning@hkbu.edu.hk

Fanxu Meng*
Peking University
fxmeng@stu.pku.edu.cn

Ruikai Zhou
Harbin Institute of Technology, Shenzhen
220110520@stu.hit.edu.cn

Michael K. NG
Hong Kong Baptist University
michael-ng@hkbu.edu.hk

Wenjie Pei
Harbin Institute of Technology, Shenzhen
wenjiecoder@outlook.com

Muhan Zhang
Peking University
muhan@pku.edu.cn

Abstract

Large Foundation Models (LFMs) have achieved remarkable success across various domains, yet their enormous parameter counts severely hinder deployment on resource-constrained devices. Recent pruning methods leverage low-rank structures in attention mechanisms, with SOTA techniques like CLOVER and Olica targeting redundancies in weight pairs (e.g., $W_Q^i(W_K^i)^\top$ from i-th attention head) to enable more aggressive compression than single-matrix pruning. However, these static approaches overlook data-driven interactions, as singular value distributions of static weight pairs differ significantly from data-driven counterparts (e.g., $XW_Q^i(W_K^i)^\top X^\top$), often yielding suboptimal performance. To address this, we introduce TransKV, a data-driven pruning framework that derives a projection matrix from keys (XW_K^i) and values (XW_V^i), seamlessly integrating it into weight-pair computations for precise low-rank truncation. TransKV natively supports core LFM components, including MHA, GQA, RMSNorm, and RoPE, ensuring compatibility with contemporary LFMs. Extensive experiments on diverse benchmarks across multiple domains validate TransKV's superiority, outperforming SOTA methods on various LFMs at equivalent compression ratios. The code is available on GitHub through: https://github.com/xuguangning1218/TransKV

1. Introduction

Large Foundation Models (LFMs) [49] have achieved remarkable success across diverse domains, including natural language processing [1, 8, 46], computer vision [14, 35, 58], and speech processing [44, 48]. These LFMs empower a wide range of applications, such as machine translation, object detection, and visual question answering, thereby driving innovation in AI-driven solutions. However, the massive scale of LFMs introduces significant challenges for deployment on resource-constrained devices, owing to their high computational demands and memory usage [10]. Pruning aims to reduce model size and redundancy while preserving accuracy [32], resulting in reduced resource costs. The pruning target for LFMs is over-parameterized attention layers [60], as they enable LFMs to distill complex

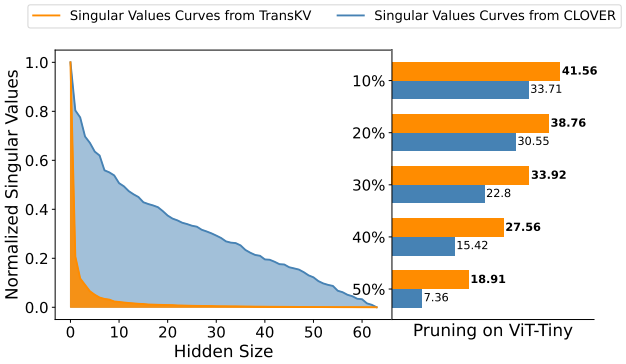


Figure 1. We illustrate one attention head in a ViT-Tiny model with ImageNet-1K (see Appendix A.1 for more ViT variants' examples). The left panel reveals a marked difference in singular value distributions between CLOVER ($W_Q^i(W_K^i)^\top$) and TransKV ($XW_Q^i(W_K^i)^\top X^\top$). The right panel shows accuracy for both methods on ViT-Tiny with ImageNet-1K, across compression ratios from 10% to 50%. These results confirm TransKV's superior pruning performance over CLOVER.

*Fanxu Meng is the corresponding author.

patterns while engendering substantial redundancy and inefficiency. These redundancies often exhibit near-low-rank properties [27], implying that much of the parameter space is underutilized and ripe for compression.

To leverage low-rank properties in attention, Singular Value Decomposition (SVD) [17] and Principal Component Analysis (PCA) [23] have become all the rage in LFMs pruning [5, 33, 63]. For SVD-based approaches, typical methods [33, 63] decompose individual attention weights to shrink attention heads by removing less important dimensions. For PCA-based methods, correlations in input features (e.g., $XX^\top$) are leveraged to compress attention heads [5]. Beyond these, the most eye-catching advancements are recent works like CLOVER [38] and Olica [21]. They pioneer new ways to capture low-rank properties through weight pairs, e.g.,$W_Q^i(W_K^i)^\top$ from i-th attention head, rather than individual weight matrices like W_Q^i. This shift is motivated by the observation that redundancies in weight-pair matrices are typically more pronounced than in single matrices. Their ideas are illustrated by

$$W_Q^i \left(W_K^i\right)^\top = U_d \Sigma_d V_d^\top. \tag{1}$$

Here, $W_Q^i, W_K^i \in \mathbb{R}^{d\times d'}$ denote the query and key weight matrices from the i-th attention head, where d is the embedding dimension and d' the attention projection dimension. Pruning is performed as follows: W_Q^i is replaced by the r-truncated $U_r\Sigma_r$, resulting in a d-by-r matrix; W_K^i is replaced by the r-truncated V_r, yielding a d-by-r matrix. Despite this insightful observation, these methods still suffer from a significant drawback: the singular value distributions of static weight-pair matrices (e.g., $W_Q^i(W_K^i)^\top$) may differ significantly from those of their data-driven counterparts (e.g., $XW_Q^i(W_K^i)^\top X^\top$), as illustrated in Figure 1. That is, pruning based on static weight-pair matrices may yield suboptimal results compared to data-driven approaches.

To address this limitation, we propose TransKV, a novel data-driven pruning framework tailored for LFMs. At its core, we derive projection matrices P_K^i, P_V^i from attention keys (XW_K^i) and values (XW_V^i) rather than relying solely on static weights (W_K^i or W_V^i) [63] or features correlations (e.g.,$XX^\top$) [5]. Once P_K^i, P_V^i are determined, it is seamlessly interposed into the attention weight-pair (e.g. $XW_Q^iP_K^i(P_K^i)^\top(W_K^i)^\top X^\top$) without altering the forward pass and similarly for value-output pairs. Pruning is then initiated by truncating P_K^i, P_V^i, tailored to specific compression objectives. To recover any potential performance degradation, we employ LoRA [24] to restore near-original accuracy on downstream tasks. To sum up, our contribution can be concluded as follows:

- We propose a data-driven pruning method named TransKV, which guides the pruning of attention weight pairs via data-dependent projection matrix, achieving superior effectiveness compared to static weight pairs methods.
- Through its data-dependent projection matrix, TransKV naturally supports GQA, RMSNorm and RoPE, thereby addressing the limitations inherent in CLOVER [38].
- Extensive experiments on LFMs (including DeepSeek-OCR) across computer vision, natural language processing, and speech processing domains demonstrate the superiority of our method on multiple benchmarks.

2. Related Work

2.1. Large Foundation Model Compression

LFMs compression techniques aim at reducing the model GPU memory and inference latency while retaining the maximum performance. Usually, they include knowledge distillation [22, 65], low-rank approximation [6, 51], parameter pruning [31, 36], and quantization [26, 66].

By effectively capitalizing on the inherent parameter redundancy within models, LFMs pruning methods [18, 28, 57] offer a distinct advantage in a single shot with low computational overhead, particularly in scenarios where subsequent fine-tuning is not required [29]. In this field, several attractive research works have emerged recently. LLM-Pruner [36] is a task-agnostic method based on gradient information, which reduces parameters by selectively removing non-critical attention heads. Beyond gradient-guided approaches, FLAP [3] employs fluctuation metrics to identify and prune weight columns in model parameters. To exploit the low-rank properties inherent in individual weight matrices, SliceGPT compresses weights by deleting rows and columns via projection matrices constructed from input data to maximize feature correlations. Furthermore, SVD-LLM [63] utilizes truncation-aware singular value decomposition on each pretrained weight matrix to compress attention modules. More recently, both CLOVER [38] and Olica [21] focus on applying SVD to pairs of attention weights, leveraging their greater redundancy compared to single weights.

2.2. Low-rank Properties in Attention

Low-rank properties in attention mechanisms indicate that pretrained weight matrices can be represented with a compact parameter structure, exposing inherent redundancies [62]. By leveraging these properties, LFMs can be effectively fine-tuned with only a small number of parameters [24, 70], achieve inference acceleration through low-rank adapted attention variants [2, 34, 53], reduce KV cache [9, 54], or undergo targeted compression [21, 38].

Generally, SVD and PCA are two popular techniques employed to exploit low-rank properties in attention compression [5, 63], primarily because they can rearrange singular values in descending order. Beyond these prevalent methods, alternative techniques for compression include Kernel-based Low-Rank (KLR) models [41], which utilize

kernel methods to capture nonlinear low-rank structures; high-order low-rank approaches, such as tensor decomposition [43], to detect low-rank patterns in high-dimensional relationships; and SoLA [25], which integrates soft activation sparsity with low-rank approximations to accelerate inference via activation-guided factorization.

3. Preliminary

3.1. Multi-Head Attention (MHA)

Multi-Head Attention (MHA) [60] is a foundational mechanism in attention. It is formalized as follows:

$$\text{MHA}(X) = \text{Concat}\left(\text{head}_1(X), \cdots, \text{head}_h(X)\right) W_O, \quad (2)$$

$$= \sum_{i=1}^{h} \text{head}_i(X) W_O^i, \quad \text{(equivalent form)} \quad (3)$$

where $X \in \mathbb{R}^{L\times d}$ is an arbitrary input sample from the dataset $\mathcal{D}$, h indicates the number of attention heads, L denotes the sequence length, and d and d' represent the model dimension and per-head dimension, respectively. The output projection matrix $W_O \in \mathbb{R}^{hd'\times d}$ is shared across heads; equivalently, $W_O^i \in \mathbb{R}^{d'\times d}$ denotes the i-th sub-matrix (slice) of W_O, projecting each head's output back to the embedding dimension d. Here, Concat$(\cdot)$ denotes the concatenation operation that stacks h head matrices, each of size $L \times d'$, along the column dimension to form a single L-by-hd' matrix, with

$$\text{head}_i(X) = \text{softmax}\left(\frac{XW_Q^i (W_K^i)^\top X^\top}{\sqrt{d'}}\right) XW_V^i, \quad (4)$$

where $W_Q^i \in \mathbb{R}^{d\times d'}$, $W_K^i \in \mathbb{R}^{d\times d'}$, $W_V^i \in \mathbb{R}^{d\times d'}$ are the i-th query, key, and value projection matrices, respectively.

3.2. Grouped-Query Attention (GQA)

Grouped Query Attention (GQA) [2] is proposed to reduce the computational cost and storage in attention mechanisms by adjusting head counts and incorporating sharing mechanisms. Specifically, the h query heads are partitioned into g groups (where $g \le h$ and h is divisible by g for even grouping), such that each group shares a single key projection matrix and a single value projection matrix. This reduces computational overhead compared to standard MHA while maintaining performance. The attention computation for the i-th head is formalized as:

$$\text{head}_i^g(X) = \text{softmax}\left(\frac{XW_Q^i (W_K^p)^\top X^\top}{\sqrt{d'}}\right) XW_V^p, \quad (5)$$

where $p = \lceil i \cdot \frac{g}{h} \rceil$ denotes the group index for the i-th head (ranging from 1 to g) and $\lceil y \rceil$ returns the smallest integer that is greater than or equal to y. This setting can ensure each $W_K^p \in \mathbb{R}^{d\times d'}$ and $W_V^p \in \mathbb{R}^{d\times d'}$ are shared among h/g query heads per group.

4. Methodology

Here, we give a full account of our TransKV as shown in Figure 2. Following this figure, we explain TransKV in a progressive process with four steps as Data-Driven Attention Weight Pair Formation, Projection Matrix Determination, TransKV Pruning on Core LMFs' Components, and TransKV for Inference.

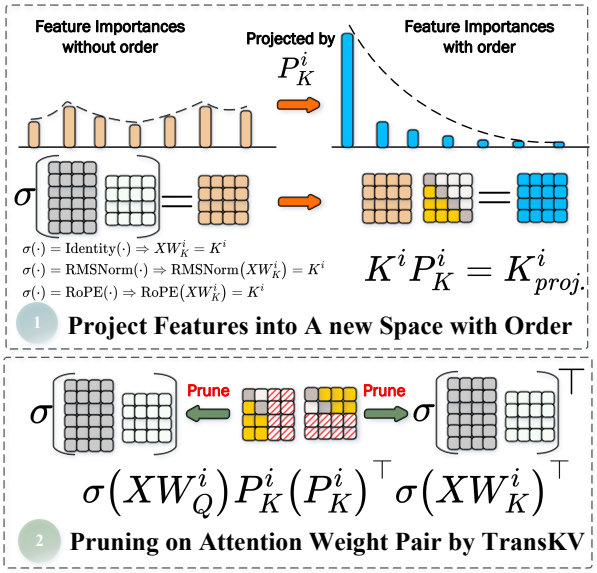


Figure 2. The TransKV data-driven pruning framework focuses on query-key pair. We first identify a projection matrix P_K^i for the i-th attention head to minimize the Frobenius norm distance $\|\sigma(XW_K^i) - \sigma(XW_K^i)P_K^i(P_K^i)^\top\|_F^2$. Then, we integrate P_K^i into the query-key pair computation as $\sigma(XW_Q^i)P_K^i(P_K^i)^\top\sigma(XW_K^i)^\top X^\top$. Subsequently, pruning is performed by using truncated projection matrix P_K^i. Finally, the model can be fine-tuned to recover performance.

4.1. Data-Driven Attention Weight Pairs Formation

The initial step of TransKV is to identify the data-driven term in attention. For simplicity, we take MHA shown in Equation (3) as an example. In this equation, we have shifted the traditional MHA equation from a concatenation operation to a summation operation. It is easy to know that they are equivalent because of the matrix multiplication principle.

Upon this setting, the per-head attention mechanism in MHA can be transformed as

$$H^i = \text{softmax}\left(\frac{\overbrace{XW_Q^i (W_K^i)^\top X^\top}^{\text{data-driven}}}{\sqrt{d'}}\right) \underbrace{XW_V^i W_O^i}_{\text{data-driven}}, \quad (6)$$

where $H^i \in \mathbb{R}^{L\times d}$ is the intermediate hidden state in i-th attention head. Consequently, the final MHA output can be shown as $\text{MHA}(X) = \sum_{i=1}^{h} H^i$. Based on Equation (6), we can identify two data-driven terms. The first is the $XW_Q^i(W_K^i)^\top X^\top$ term, which represents the input-integrated query-key weight pair. The second is the $XW_V^i W_O^i$ term, which originates from the input-integrated value-output weight pair. While recent methods [21, 38]

focus exclusively on the static weight pair terms, we propose performing pruning based on these identified data-driven attention weight pair terms.

4.2. Projection Matrix Determination

We then introduce the projection matrix determination. For simplicity, we primarily focus on demonstrating the process of finding a projection matrix P_K^i in the query-key data-driven pair. To achieve effective pruning while minimally affecting the original LFM's performance, we assume that P_K^i should satisfy two properties: First, it should project the features into a space ordered by descending singular values. Second, it should be non-intrusive, meaning it does not alter the original structure. Accordingly, we formulate an optimization problem for finding this matrix as

$$\arg\min_{P_K^i} \sum_{X\in\mathcal{D}} \|XW_K^i - XW_K^iP_K^i(P_K^i)^\top\|_F^2, \tag{7}$$

such that $(P_K^i)^\top P_K^i = I$. Here, $\mathcal{D}$ is usually a training dataset, and we set P_K^i as a d'-by-r matrix. There is a close-form solution to this problem, that is PCA. The specific manner is applying SVD on the following correlation matrix

$$C = \sum_{X\in\mathcal{D}} (XW_K^i)^\top (XW_K^i). \tag{8}$$

Then, P_K^i can be set as the first r left singular vectors from U of C. Beyond solving the projection matrix from key side, it is applicable to find P_Q^i for MHA following similar procedures. However, this is inapplicable to GQA, where the number of key heads is smaller than query heads. Finally, we can follow the similar procedures to find P_V^i or P_O^i in value-output pair.

4.3. TransKV Pruning on Core LFMs' Components

To satisfy the non-intrusive assumption, we propose to interpose the P_K^i into $XW_Q^i(W_K^i)^\top X^\top$ and P_V^i into the $XW_V^iW_O^i$.

For MHA setting, the pruning strategy for i-th attention head by TransKV can be expressed as follows:

$$\tilde{H}^i = \text{softmax}\left(\frac{X\tilde{W}_Q^i(\tilde{W}_K^i)^\top X^\top}{\sqrt{d'}}\right) X\tilde{W}_V^i\tilde{W}_O^i, \tag{9}$$

where $\tilde{W}_Q^i = W_Q^iP_K^i$, $\tilde{W}_K^i = W_K^iP_K^i$, $\tilde{W}_V^i = W_V^iP_V^i$, and $\tilde{W}_O^i = (P_V^i)^\top W_O^i$. Once the truncation operations for P_K^i and P_V^i are conducted, the pruned query, key, value weight matrices are with d-by-r size and the pruned output weight matrix is with r-by-d size. Upon this setting, our TransKV can be applied in off-line pruning without introducing runtime latency.

For GQA setting, our TransKV can naturally support GQA while CLOVER causes redundant P_K^i and P_V^i through per-head SVD [38]. The pruning on GQA can be demonstrated as follows:

$$\tilde{H}^i = \text{softmax}\left(\frac{X\tilde{W}_Q^i(\tilde{W}_K^p)^\top X^\top}{\sqrt{d'}}\right) X\tilde{W}_V^p\tilde{W}_O^i, \tag{10}$$

where $\tilde{W}_Q^i = W_Q^iP_K^p$, $\tilde{W}_K^p = W_K^pP_K^p$, $\tilde{W}_V^p = W_V^pP_V^p$, $\tilde{W}_O^i = (P_V^p)^\top W_O^i$, and $p = \lceil i\cdot\frac{g}{h}\rceil$ denotes the group index for the i-th attention head. Note that, there are h/g groups in GQA and all queries in each group share one key and its projection matrix. In the end, our TransKV can naturally support GQA without any particular adaption.

In the RMSNorm [67] and RoPE [16] setting, our TransKV inherently supports those operations, whereas CLOVER focuses solely on applying SVD to the linear static weight pairs [38]. Normally, TransKV can handle this challenge in a very general way. Suppose we are in MHA scenario, We can obtain the projection matrix through solving the following optimization problem:

$$\arg\min_{P_K^i} \sum_{X\in\mathcal{D}} \|\sigma(XW_K^i) - \sigma(XW_K^i)P_K^i(P_K^i)^\top\|_F^2, \tag{11}$$

such that $(P_K^i)^\top P_K^i = I$. Here, $\sigma(\cdot)$ can be a RMSNorm or RoPE function. Finally, the pruning strategy can be expressed as follows:

$$\tilde{H}^i = \text{softmax}\left(\frac{\widetilde{\sigma(XW_Q^i)}\widetilde{\sigma(XW_K^i)}^\top}{\sqrt{d'}}\right) X\tilde{W}_V^i\tilde{W}_O^i. \tag{12}$$

Here, $\tilde{W}_V^i = W_V^iP_V^i$ and $\tilde{W}_O^i = (P_V^i)^\top W_O^i$ are the pruned matrices without σ function. $\widetilde{\sigma(XW_Q^i)} = \sigma(XW_Q^i)P_K^i$, and $\widetilde{\sigma(XW_K^i)} = \sigma(XW_K^i)P_K^i$ are the pruned matrices enhanced by σ function. For GQA scenario, it is similar to MHA one.

4.4. TransKV for Inference

Following pruning with TransKV, direct inference for immediate deployment can be applied to LFMs. Here, we discuss the computational costs for direct LFMs inference.

To quantify the computational cost, we use arithmetic intensity, defined as $\frac{\text{FLOPs}}{\text{Memory Access}}$ [68], as the metric. Here, we mainly discuss the non-RoPE & RMSNorm and the RoPE & RMSNorm in MHA and GQA after pruning by TransKV.

For non-RoPE & RMSNorm operations, offline pruning can be applied to both query-key and value-output pairs. Then, the arithmetic intensity of MHA is $\frac{2h_qLr}{2h_qr+2h_qLr} = \frac{L}{1+L}$, which is $\mathcal{O}(1)$. The corresponding intensity for GQA is $\frac{2Lh_qr}{2h_qr+2h_{kv}Lr} = \frac{h_q}{h_{kv}}\cdot\frac{L}{h_q/h_{kv}+L}$, which is $\mathcal{O}(\frac{h_q}{h_{kv}})$. Here, L denotes the sequence length, h_q and h_{kv} represent the numbers of query heads and key-value heads, d' and r are the dimension of attention head and the pruned attention head, respectively.

For RoPE & RMSNorm operations, an online multiplication of a d'-by-r projection matrix is required only in query-key pair, while the value-output pair can still be pruned offline. To reduce inference cost, we can cache the pruned keys in KV cache, shrinking their size from $L\times d'$ to $L\times r$ and requiring computation only to the current token. Consequently, MHA's arithmetic intensity becomes $\frac{2h_qr(L+d')}{2h_qr+2h_qLr} = \frac{1+d'/L}{1+1/L} \approx 1$. TransKV's computational cost remains $\mathcal{O}(1)$. Similarly, the corresponding GQA's arithmetic intensity is $\frac{2h_qrL+h_qrd'+h_{kv}rd'}{2h_qr+2h_{kv}Lr} = \frac{h_q}{h_{kv}}\cdot\frac{2L+d'(1+h_{kv}/h_q)}{2L+2h_q/h_{kv}} \approx \frac{h_q}{h_{kv}}$, so TransKV's computational cost is still $\mathcal{O}(\frac{h_q}{h_{kv}})$.

5. Experiments

To validate TransKV, we conduct experiments on LLaMA 3 [20] and Qwen2.5 [45] for NLP, DeepSeek-OCR [64] and ViT [14] for CV, and Whisper [48] for speech processing. Experiments on TransKV are conducted on a hardware setup featuring 8 ×

NVIDIA H20 GPUs (98 GB each) and Intel(R) Xeon(R) Platinum 8558 CPUs (48 cores), utilizing PyTorch 2.8.0 with CUDA 12.8.

5.1. Main Comparison Experiment

Here, we validate TransKV on NLP. We begin by employing common-sense reasoning datasets for zero-shot classification evaluation, including BoolQ [11], PIQA [7], HellaSwag [69] (HellaS), WinoGrande [52] (WinoG), ARC-Easy [12] (ARC-e), ARC-Challenge [12] (ARC-c), and OpenBookQA [40] (OBQA). Next, we select LLaMA 3-8B [20] as the backbone LLM, given its incorporation of GQA [2] and RoPE [56]. Finally, we compare TransKV against three state-of-the-art pruning methods: LLM-Pruner [36], SliceGPT [5], and FLAP [3]. Note that we implement the baseline methods according to their original configurations, and TransKV projection matrices P_K^i, P_V^i are constructed based on the full training datasets for each downstream task.

As evident from the Table 1, TransKV consistently outperforms FLAP, SliceGPT, and LLM-Pruner across all pruning ratios in most tasks. At a 10% pruning ratio, TransKV attains an average accuracy of 64.80%, approaching the baseline, whereas the others range from 59.09% to 62.54%. This superiority becomes increasingly evident at higher ratios; for instance, at 50% pruning, TransKV sustains an average accuracy of 54.65%, markedly surpassing FLAP (50.91%), SliceGPT (34.33%), and LLM-Pruner (33.09%). Consequently, TransKV achieves the optimal compression-performance trade-off, preserving near-baseline accuracy at low ratios while minimizing degradation at high ratios.

5.2. TransKV Task Sensitivity and Recovery Ability

For task sensitivity analysis, we build TransKV's projection matrices derived from different settings. Specifically, we construct projection matrices P_K^i, P_V^i based on corresponding tasks, WikiText-2 [39], and Penn Treebank (PTB) [37], respectively. Subsequently, we apply TransKV equipped with those projection matrices to prune LLaMA 3-8B on common-sense reasoning datasets.

As shown in Figure 3 (see Appendix A.2 for results on each individual dataset), the pruning performance on task-oriented projection matrices substantially outperforms that obtained using matrices constructed from general corpora such as WikiText-2 or PTB. This result is intuitively expected, given that TransKV is explicitly designed to prune target models based on a data-driven manner. Beyond this result, we can also observe that TransKV exhibits strong capability in identifying model redundancies in testing stage through task-oriented projection matrices derived from correlated training data.

For recovery ability analysis, we employ LoRA [24] to finetune the 50% pruned LLaMA 3-8B on common-sense reasoning datasets. To ensure a fair and efficient fine-tuning setup, all pruned models from LLM-Pruner, SliceGPT, FLAP, and TransKV share the same configuration. The detailed training settings are as follows: the LoRA rank is set to 32, the learning rate is set to 2e-4, the maximum token length is 512, the training batch size is 4, the number of training steps is 125, and the prompt template follows the setting from language model evaluation harness framework [19].

Figure 4 demonstrates that TransKV achieves the highest fine-tuning performance among other methods, with an average accuracy of 59.39%—2.7% higher than the second-best method, FLAP.

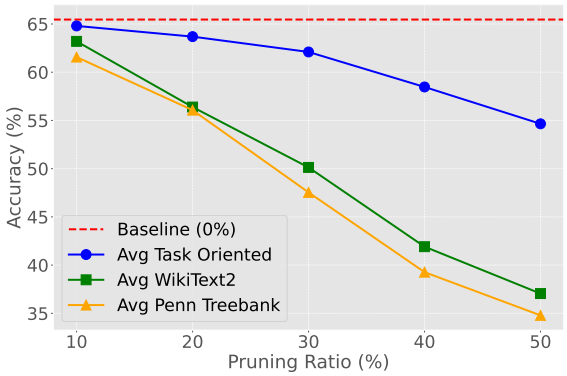


Figure 3. TransKV's task sensitivity results driven by various originating projection matrices on common-sense reasoning.

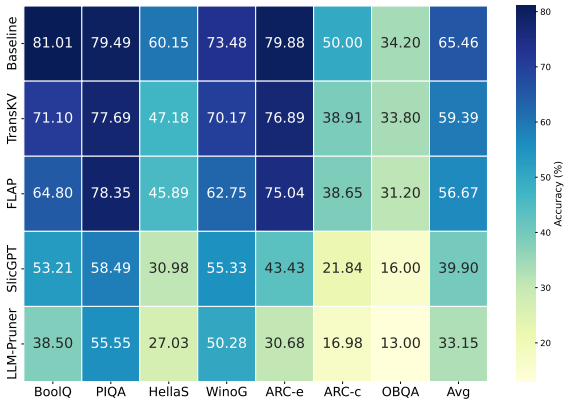


Figure 4. Finetune on 50% pruned LLaMA 3-8B through LoRA.

Beyond the average performance, TransKV also excels on individual datasets; for example, its fine-tuning performance on the WinoG dataset leads FLAP by 7.4%.

5.3. TransKV Scalability Analysis

In model scalability analysis, we choose Qwen2.5 [45] with its size scale from 3B to 14B. Then, we compare TransKV to FLAP in varying pruning ratios. In dataset scalability analysis, we choose RTE [61] for semantic reasoning task, SST2 [55] for binary sentiment classification task, and COPA [50] for commonsense reasoning. Note that, the TrasnKV's projection matrices P_V^i, P_K^i are built from the full training dataset in each task.

Table 2 (please refer to Appendix A.3 for completed results, including 0.5B and 1.5B variants) demonstrates that both methods exhibit accuracy declines as the pruning ratio increases, though the degradation is more gradual for TransKV, which consistently achieves the best values in most cases compared to FLAP. At lower pruning ratio, TransKV shows a slight superiority to FLAP in 3B model on RTE, whereas it maintains significantly higher accuracies in larger models (7B and 14B) compared to FLAP, indicating superior redundancy identification capability in larger architectures. These results show TransKV's excellence in redundancy identification, enabling superior preservation of accuracies across

Table 1. Main comparison results on LLaMA 3-8B in varying pruning ratio. The subscript of LLM-Pruner denotes its actual pruning ratio.

Method	Ratio	BoolQ	PIQA	HellaS	WinoG	ARC-e	ARC-c	OBQA	Avg.↑
Baseline	0%	81.01	79.49	60.15	73.48	79.88	50.00	34.20	65.46
SliceGPT	10%	68.07	73.56	50.65	70.80	76.05	43.69	30.80	59.09
FLAP		76.57	**79.16**	57.20	69.69	77.06	44.45	33.60	62.54
TransKV		**79.57**	78.94	**58.66**	**73.24**	**79.08**	**49.49**	**34.60**	**64.80**
SliceGPT	20%	44.53	67.41	42.50	65.75	64.81	33.28	24.80	49.01
LLM-Pruner$_{12.5\%}$		74.40	**79.05**	**57.69**	68.75	79.25	**48.55**	33.80	63.07
FLAP		71.53	77.64	54.61	66.38	74.33	41.55	32.00	59.72
TransKV		**75.23**	78.07	56.51	**73.24**	**79.59**	48.38	**34.80**	**63.69**
SliceGPT	30%	37.83	61.81	35.39	61.88	50.29	24.23	21.60	41.86
LLM-Pruner$_{25\%}$		62.05	76.12	47.42	55.25	70.29	34.56	19.20	52.13
FLAP		68.50	77.15	52.33	63.77	73.23	37.71	31.40	57.73
TransKV		**71.04**	**77.20**	**53.15**	**73.32**	**77.99**	**46.42**	**35.60**	**62.10**
SliceGPT	40%	37.83	57.67	30.71	57.70	41.20	19.45	15.40	37.14
LLM-Pruner$_{37.5\%}$		44.19	55.98	27.68	50.20	28.79	18.94	14.80	34.37
FLAP		**64.50**	**76.06**	**49.93**	61.96	71.04	35.24	28.80	55.36
TransKV		61.16	75.68	48.73	**68.67**	**76.47**	**43.60**	**35.00**	**58.47**
SliceGPT	50%	37.83	55.06	28.46	52.57	35.44	18.17	12.80	34.33
LLM-Pruner$_{50\%}$		39.36	54.73	26.85	48.93	27.48	19.88	14.40	33.09
FLAP		**59.20**	73.72	**43.80**	57.62	65.11	31.31	25.60	50.91
TransKV		56.02	**74.70**	43.01	**64.01**	**73.86**	**38.74**	**32.20**	**54.65**

Table 2. Scalability test on Qwen2.5. FLAP's performance is shown in gray color and TransKV's performance is shown in black color. The **bold** value denotes the best accuracy.

Size	Ratio	RTE	SST2	COPA
3B	0%	75.45	90.14	85.00
	10%	75.81/**81.23**	89.22/**89.56**	**84.00**/83.00
	20%	**79.78**/78.34	**88.76**/74.66	**84.00**/83.00
	30%	**74.73**/71.12	**89.45**/69.04	79.00/**86.00**
	40%	62.82/**71.12**	**88.30**/85.55	80.00/**83.00**
	50%	**57.40**/53.79	**87.27**/48.74	**76.00**/66.00
7B	0%	81.59	91.86	91.00
	10%	**80.87**/79.06	91.63/**91.86**	**90.00**/87.00
	20%	**80.51**/77.26	**91.74**/89.11	89.00/**90.00**
	30%	**80.14**/71.12	**90.25**/80.39	91.00/**92.00**
	40%	**76.17**/55.96	**90.37**/64.68	**86.00**/85.00
	50%	**69.31**/52.35	**85.21**/51.03	**83.00**/60.00
14B	0%	80.14	89.56	90.00
	10%	**80.87**/79.42	90.25/**91.17**	**91.00**/90.00
	20%	**81.23**/77.62	90.71/**91.51**	**91.00**/**91.00**
	30%	79.78/**80.51**	**90.83**/80.85	**89.00**/88.00
	40%	**77.26**/69.31	**90.71**/54.24	**89.00**/87.00
	50%	**60.65**/53.07	**87.61**/49.31	**88.00**/55.00

varying sizes of models and at elevated pruning ratios.

5.4. Speed Test on TransKV-pruned LLaMA 3-8B

In Subsection 4.4, we have showed that TransKV-pruned GQA incurs equivalent computational costs on RoPE setting compared to Non-RoPE setting. Here, we conduct a speed test to assess TransKV pruning benefits at 50% compression versus original LLaMA 3-8B on a speed testing benchmark from HuggingFace Transformers toolkit. Experiments include 3 warm-up and 10 measurement iterations, tracking GPU performance across batch sizes and sequence lengths with fixed generated tokens. We use scale dot product attention with Flash Attention backend in non-compiled, non-kernelized generation. As shown in Figure 5, the pruned model underperforms on short sequences due to RoPE pruning overhead on 8 batch sizes setting, but shows gains as these increase, benefiting from pruned keys in the KV cache.

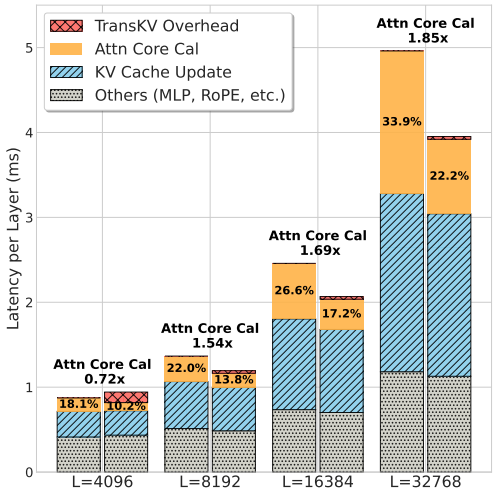


Figure 5. Inference Latency Breakdown on Batch Size = 8

5.5. TransKV Plug and Play Analysis

Here, TransKV is considered as a plug-and-play component which can be integrated with other pruning method. To conduct the ex-

Table 3. Edit distances on OmniDocBench in Gundam mode using DeepSeek-OCR. Baseline‡ denotes the reproduced results by us.

Ratio	Model	Book	Slides	Fin. Report	Textbook	Exam	Magazine	Acad. Papers	Notes	Newspaper	Overall↓
0%	Baseline‡	0.032	0.093	0.023	0.102	0.099	0.055	0.038	0.157	0.105	0.083
10%	CLOVER	**0.035**	**0.071**	**0.022**	**0.095**	0.099	0.058	**0.038**	0.167	**0.112**	0.081
	TransKV	0.036	0.074	**0.022**	**0.095**	**0.098**	**0.057**	0.040	**0.151**	0.117	**0.080**
20%	CLOVER	**0.035**	**0.081**	0.028	**0.094**	0.105	0.071	0.047	**0.151**	**0.110**	**0.084**
	TransKV	0.036	0.082	**0.022**	0.113	**0.094**	**0.065**	**0.040**	0.159	0.116	0.085
30%	CLOVER	**0.034**	**0.084**	**0.043**	0.098	0.110	0.078	**0.042**	**0.162**	**0.119**	**0.089**
	TransKV	0.035	0.087	0.044	**0.094**	**0.104**	**0.064**	0.050	0.170	0.123	**0.089**
40%	CLOVER	**0.037**	0.099	**0.043**	**0.098**	**0.106**	0.088	**0.037**	**0.166**	0.148	0.095
	TransKV	**0.037**	**0.096**	0.046	0.113	0.116	**0.064**	**0.036**	0.173	**0.133**	**0.094**
50%	CLOVER	0.038	0.111	**0.043**	**0.101**	**0.110**	0.080	0.049	0.195	0.177	0.105
	TransKV	**0.035**	**0.091**	0.050	0.103	0.114	**0.070**	**0.043**	**0.182**	**0.153**	**0.097**

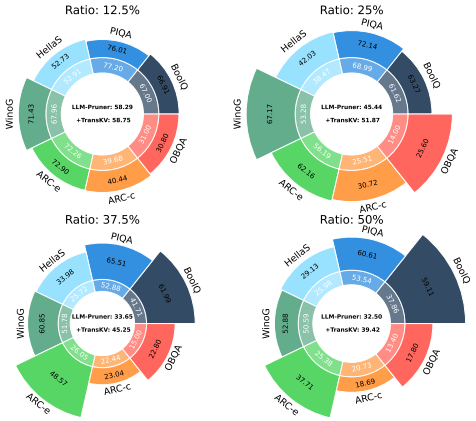


Figure 6. Performance of inner-ring is the LLM-Pruner and performance of outer-ring is the TransKV enhanced LLM-Pruner.

periment, we utilize common-sense reasoning datasets as a benchmark and LLaMA 3-8B as the backbone model. We then integrate TransKV into LLM-Pruner by replacing its original attention pruning methods. Note that other components, such as MLP, are pruned according to their original settings.

Figure 6 presents the results for LLM-Pruner and TransKV enhanced LLM-Pruner (see Appendix A.5 for other integration results). As evident from the figure, the TransKV enhanced LLM-Pruner exhibits significant performance improvements, particularly at higher pruning ratios (e.g., 50%). For the ARC-e subset, it even demonstrates a remarkable performance increase, rising from 26.05% to 48.57% after enhancement at the 37.5% pruning ratio. These results demonstrate that TransKV can serve as a plug-and-play component in LLM pruning, enhancing performance through its data-driven redundancy identification capability.

5.6. TransKV on DeepSeek-OCR

Here, we evaluate TransKV on the recently released DeepSeek-OCR model [64], which is an open-source vision model from DeepSeek-AI that enables efficient extraction of structured text, formulas, and tables from complex documents. In here, we primarily focus on pruning the DeepSeek-OCR CLIP [47] encoder, which employs MHA attention. To facilitate the experiments, we reproduce the baseline and denote it as Baseline‡. We select CLOVER as the representative static weight-pair pruning method. We then choose OmniDocBench [42] as the dataset that is a benchmark for assessing diverse document parsing in real-world scenarios. Table 3 presents the results for CLOVER and TransKV. TransKV slightly outperforms CLOVER at compression ratios of 10% to 40%, possibly because redundancies are sufficiently large for static methods. However, TransKV significantly outperforms CLOVER at a 50% ratio, highlighting its advantage in high ratio redundancies identification through data-driven manner.

5.7. Ablation Study on Projection Matrices

In this subsection, we investigate the pruning performance based on different types of projection matrices. First, we select Cifar100 [30] as the benchmark dataset. Then, we perform full fine-tuning on the ViT-Base [15] across the selected dataset. After that, we construct the projection matrices, i.e., P_Q^i, P_K^i, P_V^i, and P_O^i, for the ViT model. Finally, we combine those projection matrices and initiate pruning using TransKV for pruning. Note that we only present the Cifar100 result on value-output pair. For completed query-key and value-output results (including Cifar10 [30] and iNaturalist [59]), please refer to Appendix A.6.

The result is shown in Figure 7. We observe that the performances pruning by P_V^i are significant better than those by P_O^i and $P_V^i(P_V^i)^\top P_O^i$. As the pruning ratios increase, the differences among them become more pronounced, indicating that the choice of projection matrix plays a crucial role in maintaining performance under higher compression. In most cases across different pruning ratios, the projection P_V^i demonstrates superior pruning effectiveness, likely due to its focus on value-specific redundancies that are more critical in preserving model accuracy.

5.8. Sampling Projection Matrices Impact on ViTs

As the amount of training data becomes large, it is more applicable to construct projection matrix by employing a sampling strategy rather than using the whole dataset. To validate the effectiveness of this strategy, we utilize the ImageNet-1K [13] benchmark as it contains approximately 1.28 million images in the training set, making it highly suitable for implementing a data sampling strategy. Accordingly, we select ViT [15] as the backbone model. Then, we compare TransKV with CLOVER across different sizes of ViTs under varying pruning ratios in terms of Top-1 accuracy.

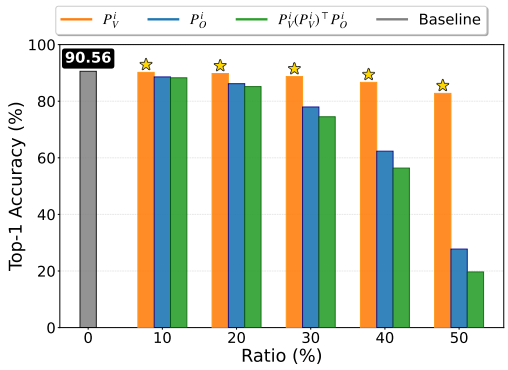


Figure 7. Ablation study on different projection matrices in query-key pair in Cifar100. Performance denotes as P_V^i means W_V^i, W_O^i are pruned by P_V^i as $X\tilde{W}_V^i\tilde{W}_O^i = XW_V^iP_V^i(P_V^i)^\top W_O^i$.

The model variants include ViT-Tiny with 5.7M parameters, ViT-Base with 86M parameters, and ViT-Large with 307M parameters. The projection matrices P_Q^i, P_V^i are computed using 1%, 5%, and 10% of the training data from each class label.

As evident from the Table 4, the sampling ratio has a limited impact on the construction of projection matrices within the selected ratios; increasing the data proportion from 5% to 10% does not guarantee a consistent performance improvement. Furthermore, TransKV and its variants outperform CLOVER across all pruning ratios for ViT-Tiny, as this variant has less redundancy compared to the others. For the Base and Large ViT variants, TransKV does not perform as well at low pruning ratios, where the redundancy is greater than in the Tiny version, and static weight-based methods suffice for redundancy identification. However, as the pruning ratio increases to higher levels, TransKV surpasses CLOVER, demonstrating its superior redundancy identification capability at higher pruning ratio.

5.9. TransKV for Speech Processing

Here, we evaluate TransKV on speech processing tasks. First, we select Common Voice v17 [4] (English) as the benchmark dataset. In this setup, we employ Whisper-small [48], a lightweight variant of OpenAI's multilingual automatic speech recognition (ASR) model with approximately 244 million parameters. Then, we adopt a pre-trained Whisper-small model on English corpora to adapt it to domain-specific acoustics. Subsequently, we apply pruning using baselines, including Taylor, and CLOVER [38], and compare their performance against TransKV in terms of Word Error Rate (WER) and Character Error Rate (CER). Specifically, all methods are applied exclusively to the query, key, value, and output weight matrices. The projection matrices P_Q^i, P_V^i of TransKV are computed using 5% of the training data from each speaker.

Table 5 presents the result on the English sub-task (see Appendix A.7 for completed results, including Cantonese and more comparison methods) applied to Whisper's Encoder (Enc.), Decoder (Dec.), and Encoder-Decoder (Enc.-Dec.), respectively. As shown, all pruning methods exhibit a descending performance curve as the pruning ratio increases from 10% to 50%, since removing parameters from the model inevitably impacts its original performance. However, our TransKV maintains superior performance among the comparison methods. In particular, TransKV significantly outperforms other methods in Encoder component pruning, as it preserves relatively reasonable WER and CER even at a 50% pruning ratio, whereas models pruned by other methods have already collapsed significantly. For the other pruned components, TransKV also demonstrates remarkable performance compared to other methods across different pruning ratios.

Table 4. Accuracy of projection matrices built with varying sampling ratios on ViTs using ImageNet-1K. TransKV superscripts denote the training data proportion for projection matrix construction. Bold indicates best among CLOVER and TransKV variants; † marks top result among TransKV sampling ratios.

Model	Ratio	CLOVER	TransKV^1	TransKV^5	TransKV^{10}
ViT-T	0%	43.14			
	10%	33.71	**41.50**	**41.56**†	**41.55**
	20%	30.55	**38.74**	**38.76**†	**38.75**
	30%	22.80	**33.89**	**33.92**†	**33.89**
	40%	15.42	**27.62**†	**27.56**	**27.60**
	50%	7.36	**18.90**	**18.91**†	**18.91**†
ViT-B	0%	73.80			
	10%	**72.61**	72.02	72.10	72.13†
	20%	**71.51**	70.52	70.55†	70.54
	30%	**68.29**	67.87	67.88	67.93†
	40%	63.20	**64.08**	**64.09**†	**64.07**
	50%	51.99	**56.27**	**56.45**†	**56.40**
ViT-L	0%	77.49			
	10%	77.13	**77.15**	**77.17**†	**77.16**
	20%	**76.85**	76.52	76.55†	76.53
	30%	**75.79**	75.33	75.35†	75.34
	40%	**74.37**	73.68	73.72	73.75†
	50%	**70.92**	70.76	70.77†	70.74
	60%	62.41	**63.26**	**63.37**	**63.39**†

Table 5. English automatic speech recognition (ASR) results on Common Voice v17 benchmark.

	Ratio	Enc.		Dec.		Enc.-Dec.	
		WER	CER	WER	CER	WER	CER
Baseline	0%	21.4	9.5	21.4	9.5	21.4	9.5
Taylor	10%	23.7	11.1	28.9	14.8	30.7	16.0
CLOVER	10%	24.2	11.4	86.7	74.6	89.1	74.3
TransKV	10%	**21.8**	**9.8**	**21.6**	**9.7**	**21.9**	**9.9**
Taylor	20%	33.2	17.7	340.6	227.5	86.6	72.3
CLOVER	20%	28.5	14.6	173.7	156.0	230.0	203.4
TransKV	20%	**22.3**	**10.1**	**22.0**	**10.0**	**22.5**	**10.3**
Taylor	30%	83.5	61.6	119.9	103.8	99.7	98.0
CLOVER	30%	60.9	45.5	122.0	104.4	157.0	132.0
TransKV	30%	**23.4**	**10.9**	**24.4**	**11.6**	**25.8**	**12.8**
Taylor	40%	157.7	158.9	96.9	97.2	100.1	410.9
CLOVER	40%	149.7	131.0	93.4	93.6	107.3	101.3
TransKV	40%	**26.4**	**13.1**	**35.2**	**19.4**	**35.5**	**18.8**
Taylor	50%	1227.3	1195.0	98.4	168.6	100.0	98.3
CLOVER	50%	205.4	180.7	**96.9**	96.6	99.8	101.3
TransKV	50%	**38.5**	**23.2**	106.4	**82.1**	**85.5**	**60.6**

6. Conclusion

In conclusion, TransKV advances the data-driven pruning framework for LFMs (including the recent DeepSeek-OCR) by embedding projection matrices into attention query-key (Q-K) and value-output (V-O) pairs, enabling low-rank truncation that aligns with real-world data distributions. This approach not only outperforms static methods like CLOVER in accuracy retention but also natively supports GQA, RMSNorm and RoPE, facilitating deployment on modern LFMs. Extensive experiments across CV, NLP, and speech processing benchmarks confirm its superiority at equivalent compression ratios. For future work, we plan to extend support to more attention variants and multimodal LFMs.

References

[1] Josh Achiam, Steven Adler, Sandhini Agarwal, Lama Ahmad, Ilge Akkaya, Florencia Leoni Aleman, Diogo Almeida, Janko Altenschmidt, Sam Altman, Shyamal Anadkat, et al. Gpt-4 technical report. *arXiv preprint arXiv:2303.08774*, 2023. 1

[2] Joshua Ainslie, James Lee-Thorp, Michiel De Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. Gqa: Training generalized multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245*, 2023. 2, 3, 5

[3] Yongqi An, Xu Zhao, Tao Yu, Ming Tang, and Jinqiao Wang. Fluctuation-based adaptive structured pruning for large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 10865–10873, 2024. 2, 5

[4] Rosana Ardila, Megan Branson, Kelly Davis, Michael Kohler, Josh Meyer, Michael Henretty, Reuben Morais, Lindsay Saunders, Francis Tyers, and Gregor Weber. Common voice: A massively-multilingual speech corpus. In *Proceedings of the Twelfth Language Resources and Evaluation Conference*, pages 4218–4222, 2020. 8

[5] Saleh Ashkboos, Maximilian L Croci, Marcelo Gennari do Nascimento, Torsten Hoefler, and James Hensman. Slicegpt: Compress large language models by deleting rows and columns. In *The Twelfth International Conference on Learning Representations*, 2024. 2, 5

[6] Iz Beltagy, Matthew E Peters, and Arman Cohan. Longformer: The long-document transformer. *arXiv preprint arXiv:2004.05150*, 2020. 2

[7] Yonatan Bisk, Rowan Zellers, Jianfeng Gao, Yejin Choi, et al. Piqa: Reasoning about physical commonsense in natural language. In *Proceedings of the AAAI conference on artificial intelligence*, pages 7432–7439, 2020. 5

[8] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020. 1

[9] Chi-Chih Chang, Wei-Cheng Lin, Chien-Yu Lin, Chong-Yan Chen, Yu-Fang Hu, Pei-Shuo Wang, Ning-Chi Huang, Luis Ceze, Mohamed S Abdelfattah, and Kai-Chiang Wu. Palu: Kv-cache compression with low-rank projection. In *The Thirteenth International Conference on Learning Representations*, 2025. 2

[10] Arnav Chavan, Raghav Magazine, Shubham Kushwaha, Mérouane Debbah, and Deepak Gupta. Faster and lighter llms: A survey on current challenges and way forward. *arXiv preprint arXiv:2402.01799*, 2024. 1

[11] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. Boolq: Exploring the surprising difficulty of natural yes/no questions. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, pages 2924–2936, 2019. 5

[12] Peter Clark, Isaac Cowhey, Oren Etzioni, Tushar Khot, Ashish Sabharwal, Carissa Schoenick, and Oyvind Tafjord. Think you have solved question answering? try arc, the ai2 reasoning challenge. *arXiv preprint arXiv:1803.05457*, 2018. 5

[13] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009. 7

[14] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020. 1, 4

[15] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale. In *International Conference on Learning Representations*, 2021. 7

[16] Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha Letman, Akhil Mathur, Alan Schelten, Amy Yang, Angela Fan, et al. The llama 3 herd of models. *arXiv e-prints*, pages arXiv–2407, 2024. 4

[17] Carl Eckart and Gale Young. The approximation of one matrix by another of lower rank. *Psychometrika*, 1(3):211–218, 1936. 2

[18] Elias Frantar and Dan Alistarh. Sparsegpt: Massive language models can be accurately pruned in one-shot. In *International conference on machine learning*, pages 10323–10337. PMLR, 2023. 2

[19] Leo Gao, Jonathan Tow, Baber Abbasi, Stella Biderman, Sid Black, Anthony DiPofi, Charles Foster, Laurence Golding, Jeffrey Hsu, Alain Le Noac'h, Haonan Li, Kyle McDonell, Niklas Muennighoff, Chris Ociepa, Jason Phang, Laria Reynolds, Hailey Schoelkopf, Aviya Skowron, Lintang Sutawika, Eric Tang, Anish Thite, Ben Wang, Kevin Wang, and Andy Zou. The language model evaluation harness, 2024. 5

[20] Aaron Grattafiori, Abhimanyu Dubey, Abhinav Jauhri, Abhinav Pandey, Abhishek Kadian, Ahmad Al-Dahle, Aiesha

Letman, Akhil Mathur, Alan Schelten, Alex Vaughan, et al. The llama 3 herd of models. *arXiv preprint arXiv:2407.21783*, 2024. 4, 5

[21] Jiujun He and Huazhen Lin. Olica: Efficient structured pruning of large language models without retraining. In *International Conference on Machine Learning*, 2025. 2, 3

[22] G Hinton. Distilling the knowledge in a neural network. In *Deep Learning and Representation Learning Workshop in Conjunction with NIPS*, 2014. 2

[23] Harold Hotelling. Analysis of a complex of statistical variables into principal components. *Journal of educational psychology*, 24(6):417, 1933. 2

[24] Edward J Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, Weizhu Chen, et al. Lora: Low-rank adaptation of large language models. *ICLR*, 1(2):3, 2022. 2, 5

[25] Xinhao Huang, You-Liang Huang, and Zeyi Wen. Sola: Leveraging soft activation sparsity and low-rank decomposition for large language model compression. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 17494–17502, 2025. 3

[26] Benoit Jacob, Skirmantas Kligys, Bo Chen, Menglong Zhu, Matthew Tang, Andrew Howard, Hartwig Adam, and Dmitry Kalenichenko. Quantization and training of neural networks for efficient integer-arithmetic-only inference. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2704–2713, 2018. 2

[27] AJAY KUMAR JAISWAL, Yifan Wang, Lu Yin, Shiwei Liu, Runjin Chen, Jiawei Zhao, Ananth Grama, Yuandong Tian, and Zhangyang Wang. From low rank gradient subspace stabilization to low-rank weights: Observations, theories, and applications. In *Forty-second International Conference on Machine Learning*, 2025. 2

[28] Bo-Kyeong Kim, Geonmin Kim, Tae-Ho Kim, Thibault Castells, Shinkook Choi, Junho Shin, and Hyoung-Kyu Song. Shortened LLaMA: A simple depth pruning for large language models. In *ICLR 2024 Workshop on Mathematical and Empirical Understanding of Foundation Models*, 2024. 2

[29] Gun Il Kim, Sunga Hwang, and Beakcheol Jang. Efficient compressing and tuning methods for large language models: A systematic literature review. *ACM Computing Surveys*, 57 (10):1–39, 2025. 2

[30] Alex Krizhevsky. Learning multiple layers of features from tiny images. Technical report, University of Toronto, 2009. 7

[31] Yann LeCun, John Denker, and Sara Solla. Optimal brain damage. *Advances in neural information processing systems*, 2, 1989. 2

[32] Guanchen Li, Yixing Xu, Zeping Li, Ji Liu, Xuanwu Yin, Dong Li, and Emad Barsoum. T\'yr-the-pruner: Unlocking accurate 50% structural pruning for llms via global sparsity distribution optimization. *arXiv preprint arXiv:2503.09657*, 2025. 1

[33] Yixiao Li, Yifan Yu, Qingru Zhang, Chen Liang, Pengcheng He, Weizhu Chen, and Tuo Zhao. Losparse: Structured compression of large language models based on low-rank and sparse approximation. In *International Conference on Machine Learning*, pages 20336–20350. PMLR, 2023. 2

[34] Aixin Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Dengr, Chong Ruan, Damai Dai, Daya Guo, et al. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434*, 2024. 2

[35] Ze Liu, Yutong Lin, Yue Cao, Han Hu, Yixuan Wei, Zheng Zhang, Stephen Lin, and Baining Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 10012–10022, 2021. 1

[36] Xinyin Ma, Gongfan Fang, and Xinchao Wang. Llm-pruner: On the structural pruning of large language models. *Advances in neural information processing systems*, 36:21702–21720, 2023. 2, 5

[37] Mary Ann Marcinkiewicz. Building a large annotated corpus of english: The penn treebank. *Using Large Corpora*, 273: 31, 1994. 5

[38] Fanxu Meng, Pingzhi Tang, Fan Jiang, and Muhan Zhang. CLOVER: Cross-layer orthogonal vectors pruning. In *International Conference on Machine Learning*, 2025. 2, 3, 4, 8

[39] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations*, 2017. 5

[40] Todor Mihaylov, Peter Clark, Tushar Khot, and Ashish Sabharwal. Can a suit of armor conduct electricity? a new dataset for open book question answering. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*, pages 2381–2391, 2018. 5

[41] Ukash Nakarmi, Yanhua Wang, Jingyuan Lyu, Dong Liang, and Leslie Ying. A kernel-based low-rank (klr) model for low-dimensional manifold recovery in highly accelerated dynamic mri. *IEEE transactions on medical imaging*, 36(11): 2297–2307, 2017. 2

[42] Linke Ouyang, Yuan Qu, Hongbin Zhou, Jiawei Zhu, Rui Zhang, Qunshu Lin, Bin Wang, Zhiyuan Zhao, Man Jiang, Xiaomeng Zhao, et al. Omnidocbench: Benchmarking diverse pdf document parsing with comprehensive annotations. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 24838–24848, 2025. 7

[43] Anh-Huy Phan, Konstantin Sobolev, Konstantin Sozykin, Dmitry Ermilov, Julia Gusak, Petr Tichavskỳ, Valeriy Glukhov, Ivan Oseledets, and Andrzej Cichocki. Stable low-rank tensor decomposition for compression of convolutional neural network. In *European Conference on Computer Vision*, pages 522–539. Springer, 2020. 3

[44] Vadim Popov, Ivan Vovk, Vladimir Gogoryan, Tasnima Sadekova, and Mikhail Kudinov. Grad-tts: A diffusion probabilistic model for text-to-speech. In *International conference on machine learning*, pages 8599–8608. PMLR, 2021. 1

[45] Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le

Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2025. 4, 5

[46] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei, Ilya Sutskever, et al. Language models are unsupervised multitask learners. *OpenAI blog*, 1(8):9, 2019. 1

[47] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PmLR, 2021. 7

[48] Alec Radford, Jong Wook Kim, Tao Xu, Greg Brockman, Christine McLeavey, and Ilya Sutskever. Robust speech recognition via large-scale weak supervision. In *International conference on machine learning*, pages 28492–28518. PMLR, 2023. 1, 4, 8

[49] Vipula Rawte, Amit Sheth, and Amitava Das. A survey of hallucination in large foundation models. *arXiv preprint arXiv:2309.05922*, 2023. 1

[50] Melissa Roemmele, Cosmin Adrian Bejan, and Andrew S Gordon. Choice of plausible alternatives: An evaluation of commonsense causal reasoning. In *AAAI spring symposium: logical formalizations of commonsense reasoning*, pages 90–95, 2011. 5

[51] Rajarshi Saha, Varun Srivastava, and Mert Pilanci. Matrix compression via randomized low rank and low precision factorization. *Advances in Neural Information Processing Systems*, 36:18828–18872, 2023. 2

[52] Keisuke Sakaguchi, Ronan Le Bras, Chandra Bhagavatula, and Yejin Choi. Winogrande: An adversarial winograd schema challenge at scale. *Communications of the ACM*, 64 (9):99–106, 2021. 5

[53] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019. 2

[54] Prajwal Singhania, Siddharth Singh, Shwai He, Soheil Feizi, and Abhinav Bhatele. Loki: Low-rank keys for efficient sparse attention. *Advances in Neural Information Processing Systems*, 37:16692–16723, 2024. 2

[55] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D Manning, Andrew Y Ng, and Christopher Potts. Recursive deep models for semantic compositionality over a sentiment treebank. In *Proceedings of the 2013 conference on empirical methods in natural language processing*, pages 1631–1642, 2013. 5

[56] Jianlin Su, Murtadha Ahmed, Yu Lu, Shengfeng Pan, Wen Bo, and Yunfeng Liu. Roformer: Enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024. 5

[57] Mingjie Sun, Zhuang Liu, Anna Bair, and J Zico Kolter. A simple and effective pruning approach for large language models. In *The Twelfth International Conference on Learning Representations*, 2024. 2

[58] Hugo Touvron, Matthieu Cord, Matthijs Douze, Francisco Massa, Alexandre Sablayrolles, and Hervé Jégou. Training data-efficient image transformers & distillation through attention. In *International conference on machine learning*, pages 10347–10357. PMLR, 2021. 1

[59] Grant Van Horn, Oisin Mac Aodha, Yang Song, Yin Cui, Chen Sun, Alex Shepard, Hartwig Adam, Pietro Perona, and Serge Belongie. The inaturalist species classification and detection dataset. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 8769–8778, 2018. 7

[60] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017. 1, 3

[61] Alex Wang, Amanpreet Singh, Julian Michael, Felix Hill, Omer Levy, and Samuel Bowman. Glue: A multi-task benchmark and analysis platform for natural language understanding. In *Proceedings of the 2018 EMNLP Workshop BlackboxNLP: Analyzing and Interpreting Neural Networks for NLP*, page 353. Association for Computational Linguistics, 2018. 5

[62] Sinong Wang, Belinda Z Li, Madian Khabsa, Han Fang, and Hao Ma. Linformer: Self-attention with linear complexity. *arXiv preprint arXiv:2006.04768*, 2020. 2

[63] Xin Wang, Yu Zheng, Zhongwei Wan, and Mi Zhang. Svd-llm: Truncation-aware singular value decomposition for large language model compression. In *The Thirteenth International Conference on Learning Representations*, 2025. 2

[64] Haoran Wei, Yaofeng Sun, and Yukun Li. Deepseek-ocr: Contexts optical compression, 2025. 4, 7

[65] Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022. 2

[66] Yuzhuang Xu, Xu Han, Zonghan Yang, Shuo Wang, Qingfu Zhu, Zhiyuan Liu, Weidong Liu, and Wanxiang Che. Onebit: Towards extremely low-bit large language models. *Advances in Neural Information Processing Systems*, 37:66357–66382, 2024. 2

[67] An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, et al. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025. 4

[68] Ted Zadouri, Hubert Strauss, and Tri Dao. Hardware-efficient attention for fast decoding. *arXiv preprint arXiv:2505.21487*, 2025. 4

[69] Rowan Zellers, Ari Holtzman, Yonatan Bisk, Ali Farhadi, and Yejin Choi. Hellaswag: Can a machine really finish your sentence? In *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, 2019. 5

[70] Jiawei Zhao, Zhenyu Zhang, Beidi Chen, Zhangyang Wang, Anima Anandkumar, and Yuandong Tian. Galore: Memory-efficient llm training by gradient low-rank projection. In *Forty-first International Conference on Machine Learning*, 2024. 2

TransKV: A Data-Driven Pruning Method for Large Foundation Models

Supplementary Material

A.1. Low-rank Properties in ViTs

In this section, we first describe the methodology for obtaining the singular value distributions of both static weight pair (CLOVER) [8] and data-driven weight pair (TransKV). We then present additional examples of singular value distributions from the Vision Transformer (ViT) models [5] across different layers and attention heads.

Static Weight Pair (CLOVER). To compute the singular value distribution for static query-key weight pair, we focus on the pretrained query and key projection matrices (value-output pair follows the similar procedure). Let d denote the hidden dimension and d' the per-head attention dimension. For the i-th attention head, the pretrained query weight matrix $W_Q^i \in \mathbb{R}^{d\times d'}$ and key weight matrix $W_K^i \in \mathbb{R}^{d\times d'}$ are combined along the hidden dimension using an Einstein summation (Esum) operation, followed by singular value decomposition (SVD):

$$\text{Esum}('lD, ld \to Dd', W_Q^i, W_K^i) = U_{d'}\Sigma_{d'}V_{d'}. \quad (1)$$

where $U_{d'} \in \mathbb{R}^{d'\times d'}$ and $V_{d'} \in \mathbb{R}^{d'\times d'}$ are the left and right singular vector matrices, respectively, and $\Sigma_{d'} \in \mathbb{R}^{d'\times d'}$ is the diagonal matrix containing the singular values (in descending order). The diagonal elements of $\Sigma_{d'}$ are taken as the singular value distribution of the static query-key weight pair for the i-th attention head.

Data-Driven Weight Pair (TransKV). For the data-driven counterpart, consider an arbitrary input sequence $X \in \mathbb{R}^{L\times d}$, where L is the sequence length. The corresponding query and key matrices for the i-th attention head are $Q^i = XW_Q^i \in \mathbb{R}^{L\times d'}$ and $K^i = XW_K^i \in \mathbb{R}^{L\times d'}$. The data-driven singular value distribution is obtained analogously by performing SVD on the outer product projected along the sequence dimension:

$$\text{Esum}('lD, ld \to Dd', XW_Q^i, XW_K^i) = U_{d'}^{'}\Sigma_{d'}^{'}V_{d'}^{'}. \quad (2)$$

Here, the diagonal elements of $\Sigma_{d'}^{'}$ represent the singular value distribution of the data-driven query-key pair for the i-th attention head.

Visualization on ViT variants. We apply the aforementioned methodology to ViT variants, including ViT-Tiny (ViT-T), ViT-Base (ViT-B), and ViT-Large (ViT-L), focusing on layers 6 and 12 (with layer indexing starting from 1). For visualization clarity, we report results for only the first three attention heads in each model (out of 3 heads in ViT-T, 12 heads in ViT-B, and 16 heads in ViT-L). As shown in Figures 1, 2, and 3, the singular value distributions of static query-key weight pairs and their data-driven counterparts exhibit remarkable inconsistency. This phenomenon persists across different layers and attention heads in all examined ViT variants.

A.2. Task Sensitivity Completed Results

In the main paper, we present the average performance across common-sense reasoning datasets in the task sensitivity experiments. Here, we provide the complete results for each individual dataset in Table 1.

As shown in the table, TransKV driven by task-oriented projection matrices consistently and significantly outperform their counterparts driven by WT2 [9] or PTB [7] across various pruning ratios and datasets. This suggests that higher-quality, task-relevant projection matrices possess a superior ability to identify and eliminate latent redundancies compared to those derived from generic corpora. Furthermore, we observe that WT2-driven variants generally achieve better performance than PTB-driven ones. A plausible explanation is that the WT2 corpus contains substantially more commonsense knowledge than PTB, which partially compensates for its lower task specificity.

A.3. Scalability Completed Results

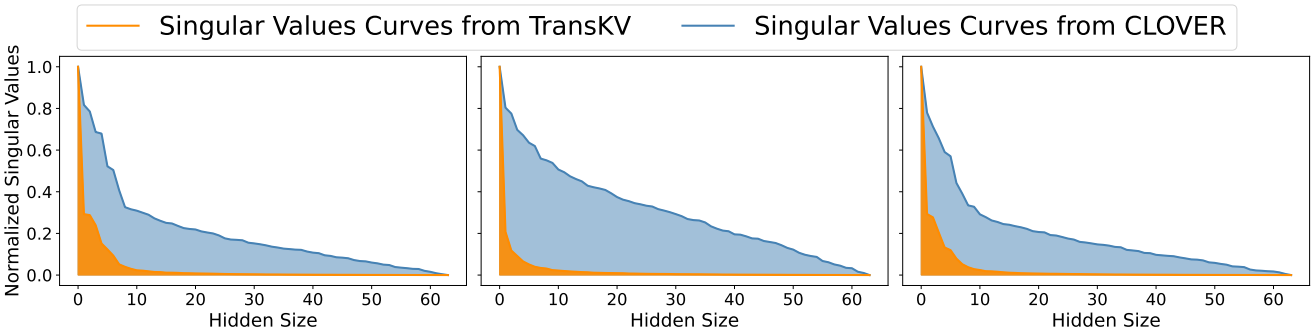
In the main paper, we demonstrate the scalability of TransKV on Qwen-2.5 models [10] ranging from 3B to 14B parameters. Here, we present the complete results across all Qwen-2.5 variants (0.5B, 1.5B, 3B, 7B, and 14B) by comparing TransKV with FLAP [1] under various pruning ratios.

As shown in Table 2, TransKV consistently and significantly outperforms FLAP across all model sizes and pruning ratios. These complete results reveal that TransKV excels at identifying both mild redundancies in smaller models (e.g., 0.5B and 1.5B) and substantial redundancies in larger models (e.g., 3B, 7B, and 14B). Moreover, TransKV exhibits particularly strong performance in high-redundancy scenarios, making it especially well-suited for compressing LFMs with massive parameter counts.

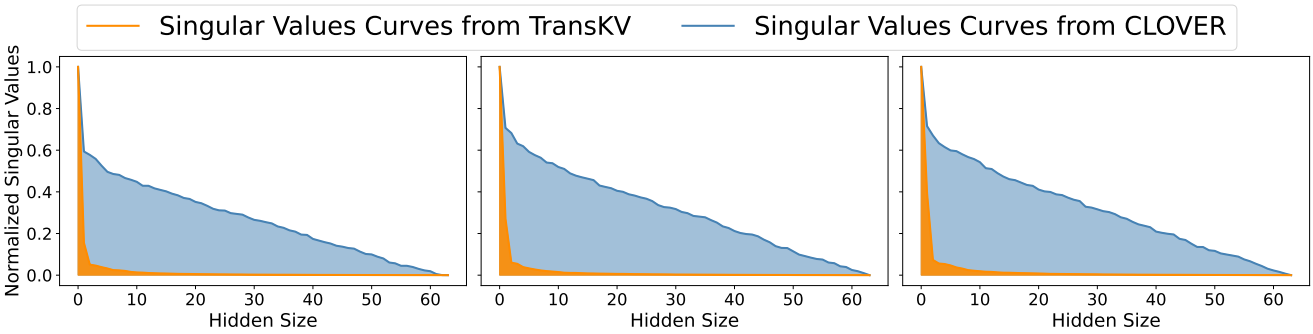
A.4. Speed Test Completed Results

In the main paper, we report the average speed test results of TransKV on a single H20 GPU. Here, we provide the complete evaluation across batch sizes ranging from 8 to 64, reporting minimum, maximum, and average values for each metric. We compare the original LLaMA-3-8B model with its 50%-pruned version produced by TransKV.

As shown in Table **??**, we measure end-to-end latency, time-to-first-token (T2F), inter-token latency (I.Tok.), and

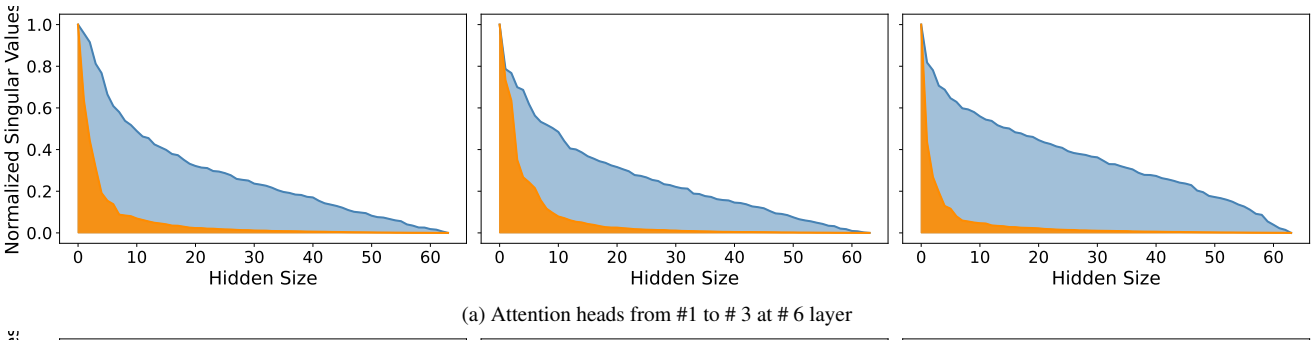


(a) Attention heads from #1 to # 3 at # 6 layer

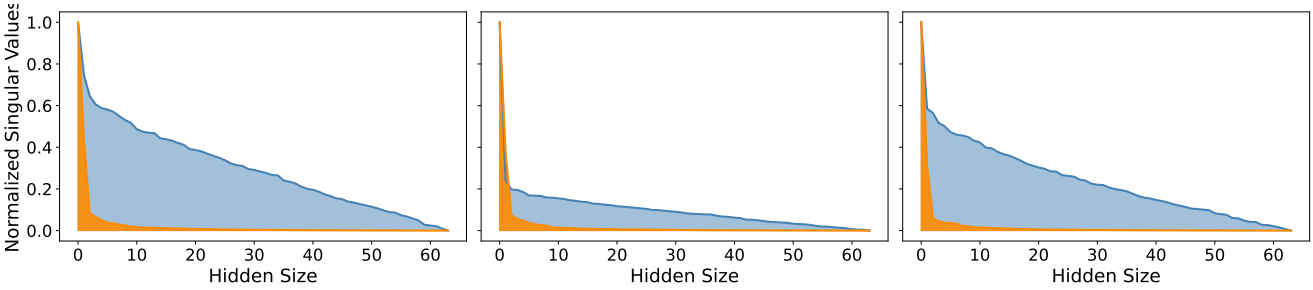


(b) Attention heads from #1 to # 3 at # 12 layer

Figure 1. ViT-T singular value distribution examples from CLOVER and TransKV

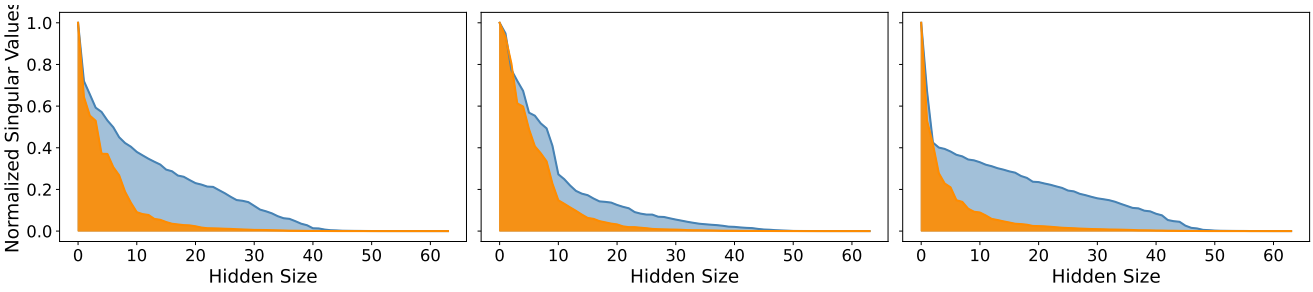


(a) Attention heads from #1 to # 3 at # 6 layer

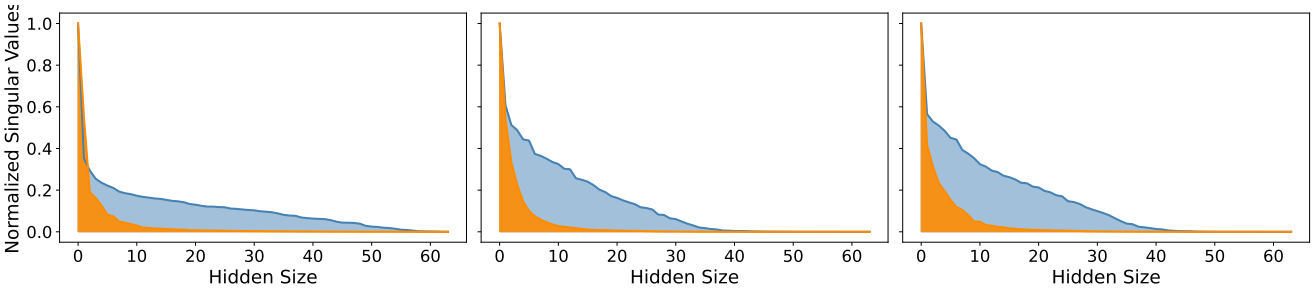


(b) Attention heads from #1 to # 3 at # 12 layer

Figure 2. ViT-B singular value distribution examples from CLOVER and TransKV



(a) ViT-L Attention heads from #1 to # 3 at # 6 layer



(b) ViT-L Attention heads from #1 to # 3 at # 12 layer

Figure 3. ViTs singular value distribution examples from CLOVER and TransKV

Table 1. TransKV task sensitivity Completed Results. Task, WT2, and PTB denote projection matrices are constructed by corresponding tasks, Wiki-text 2, and penn treebank.

Ratio	Proj.	BoolQ	PIQA	HellaS	WinoG	ARC-e	ARC-c	OBQA	Avg↑
0%	-	81.01	79.49	60.15	73.48	79.88	50.00	34.20	65.46
10%	Task	**79.57**	**78.94**	**58.66**	**73.24**	**79.08**	49.49	**34.60**	**64.80**
	WT2	73.61	78.29	56.40	72.69	78.83	**49.57**	33.00	63.20
	PTB	73.61	78.78	56.40	68.75	76.85	**49.57**	27.00	61.57
20%	Task	**75.23**	**78.07**	**56.51**	**73.24**	**79.59**	**48.38**	**34.80**	**63.69**
	WT2	61.62	75.95	49.83	64.80	75.34	43.77	23.40	56.39
	PTB	61.62	75.19	49.83	64.80	70.29	43.77	27.00	56.07
30%	Task	**71.04**	**77.20**	**53.15**	**73.32**	**77.99**	**46.42**	**35.60**	**62.10**
	WT2	50.43	72.20	41.24	58.80	68.81	35.41	24.00	50.13
	PTB	50.43	69.48	41.24	54.54	59.68	35.41	22.00	47.54
40%	Task	**61.16**	**75.68**	**48.73**	**68.67**	**76.47**	**43.60**	**35.00**	**58.47**
	WT2	40.83	66.59	33.39	52.64	58.71	23.63	17.60	41.91
	PTB	40.83	64.15	33.39	51.46	45.62	23.63	15.80	39.27
50%	Task	**56.02**	**74.70**	**43.01**	**64.01**	**73.86**	**38.74**	**32.20**	**54.65**
	WT2	38.01	60.88	28.77	51.93	45.33	19.20	15.20	37.05
	PTB	38.01	59.09	28.77	49.88	35.02	19.20	13.60	34.79

throughput (Thr.) on a single H20 GPU. As batch size and sequence length increase, TransKV consistently and substantially outperforms the original LLaMA-3-8B across all metrics. This demonstrates that the additional latency previously introduced by RoPE-aware pruning in TransKV is effectively eliminated once the batch size and sequence length exceed a certain threshold, fully realizing the acceleration benefits of KV cache compression in practical deployment.

A.5. Plug and Play Analysis Completed Results

In the main paper, we demonstrate the plug-and-play compatibility of TransKV by integrating it with LLM-Pruner. Here, we present the complete results when replacing or

Table 2. Scalability test on Qwen2.5. FLAP's performance is shown in gray color and TransKV's performance is shown in black color. The **bold** value denotes the best performance.

Size	Ratio	RTE	SST2	COPA
0.5B	0%	58.84	54.13	74.00
	10%	55.23/**55.60**	**51.38**/49.89	72.00/**74.00**
	20%	52.35/**56.32**	**57.57**/50.11	74.00/**77.00**
	30%	**53.07**/49.82	**56.08**/51.49	72.00/**73.00**
	40%	**53.43**/48.38	**51.38**/49.20	**76.00**/70.00
	50%	**49.10**/47.65	**57.11**/49.89	**72.00**/61.00
1.5B	0%	70.04	88.42	83.00
	10%	68.59/**71.84**	**89.45**/86.24	83.00/83.00
	20%	**69.68**/58.12	**89.22**/49.77	**80.00**/77.00
	30%	**64.98**/57.76	**87.61**/72.94	77.00/**81.00**
	40%	**55.23**/49.82	**86.47**/48.62	**75.00**/60.00
	50%	48.74/**50.54**	**81.77**/49.77	**68.00**/63.00
3B	0%	75.45	90.14	85.00
	10%	75.81/**81.23**	89.22/**89.56**	**84.00**/83.00
	20%	**79.78**/78.34	**88.76**/74.66	**84.00**/83.00
	30%	**74.73**/71.12	**89.45**/69.04	79.00/**86.00**
	40%	62.82/**71.12**	**88.30**/85.55	80.00/**83.00**
	50%	**57.40**/53.79	**87.27**/48.74	**76.00**/66.00
7B	0%	81.59	91.86	91.00
	10%	**80.87**/79.06	91.63/**91.86**	**90.00**/87.00
	20%	**80.51**/77.26	**91.74**/89.11	89.00/**90.00**
	30%	**80.14**/71.12	**90.25**/80.39	91.00/**92.00**
	40%	**76.17**/55.96	**90.37**/64.68	**86.00**/85.00
	50%	**69.31**/52.35	**85.21**/51.03	**83.00**/60.00
14B	0%	80.14	89.56	90.00
	10%	**80.87**/79.42	90.25/**91.17**	**91.00**/90.00
	20%	**81.23**/77.62	90.71/**91.51**	91.00/91.00
	30%	79.78/**80.51**	**90.83**/80.85	**89.00**/88.00
	40%	**77.26**/69.31	**90.71**/54.24	**89.00**/87.00
	50%	**60.65**/53.07	**87.61**/49.31	**88.00**/55.00

combining TransKV's attention pruning module with two representative structured pruning methods—FLAP [1] and SliceGPT [2]—on common-sense reasoning benchmarks (Figures 4 and 5).

As shown in Figure 4, directly substituting TransKV for FLAP's original attention pruning mechanism consistently improves performance across all evaluated pruning ratios. This confirms that TransKV's task-sensitive singular value-guided attention pruning is strictly superior to FLAP's heuristic-based approach under identical experimental conditions.

For SliceGPT (Figure 5), we retain SliceGPT's attention pruning pipeline while incorporating TransKV for attention module compression (note that SliceGPT's MLP importance scores are computed on attention outputs necessitate this sequential combination). The results reveal that TransKV boosts SliceGPT's performance at high pruning ratios (e.g., 60%–70%), whereas the improvement is marginal or even slightly negative at lower ratios. This phenomenon can be attributed to the fundamental difference in pruning granularity: SliceGPT primarily reduces the hidden dimension via its projection matrix, whereas TransKV prunes the per-head attention dimension. At low pruning ratios, attention-dimension reduction by TransKV may disturb the principal components estimated by SliceGPT on the original attention outputs, thereby altering the data distribution assumed by subsequent MLP pruning and introducing minor performance degradation. In contrast, when overall sparsity is high, the abundant redundancy in large language models allows TransKV's more accurate importance evaluation to dominate, yielding clear gains even in hybrid settings.

A.6. Ablation Study on Projection Matrices Completed Results

In the main paper, we present ablation results focusing on the value-output pairs, evaluating three candidate projection matrices: P_V^i, P_O^i, and $P_V^i(P_V^i)^\top P_O^i$. Here, we provide the complete ablation study that includes both query-key pairs (P_Q^i, P_K^i, and $P_Q^i(P_Q^i)^\top P_K^i$) and value-output pairs (P_V^i, P_O^i, and $P_V^i(P_V^i)^\top P_O^i$).

As shown in Table 4, the choice of projection matrix has a substantial impact on final pruning performance. For the value-output pair, P_V^i emerges as the optimal projection matrix for W_V^i, while its transpose $(P_V^i)^\top$ yields the best results when applied to W_O^i. In the query-key pair, P_Q^i and P_K^i exhibit nearly identical effectiveness when used to prune W_Q^i, and correspondingly, their transposes perform comparably on W_K^i. Consequently, we recommend using P_K^i to project W_Q^i and $(P_K^i)^\top$ to project W_K^i. This symmetric strategy based on the key projection matrix is not only empirically strong but also universally applicable—it works seamlessly for both MHA and GQA architectures.

A.7. Compare to EigenAttention and MatryoshkaKV

Commonality & Distinction. While TransKV shares prune-then-recover framework with mentioned papers, our core contribution lies in a data-driven, training-free efficiency that fundamentally differs from the baselines.

Difference from EigenAttention (PCA). **1. Task-Specific vs. Generic**. EigenAttention derives projections from generic corpora via SVD. In contrast, TransKV is data-driven, constructing projection matrices from task-specific data to better capture task-relevant features (see Table 3). **2. Peak Memory Efficiency**. EigenAttention reconstructs the pruned Key during attention, causing full-rank calculation overhead. TransKV operates on RoPE(K) to conduct online pruning, strictly maintaining low-rank calculations and significantly reducing peak memory.

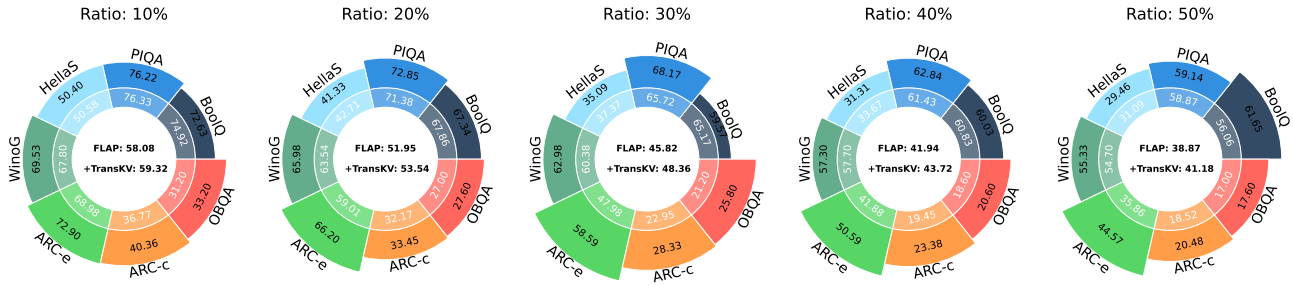


Figure 4. TransKV plugs into FLAP. Performance of inner- ring is the original FLAP and performance of outer-ring is the TransKV enhanced FLAP.

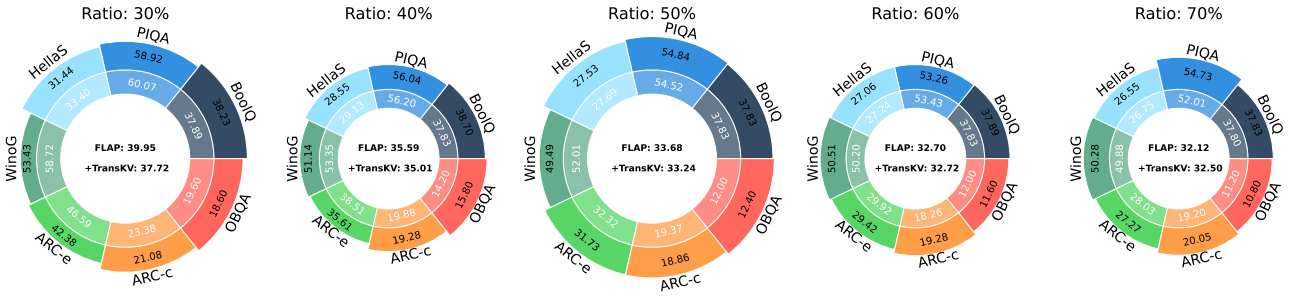


Figure 5. TransKV plugs into SliceGPT. Performance of inner- ring is the original SliceGPT and performance of outer-ring is the TransKV enhanced SliceGPT.

Table 3. Results on BoolQ, PIQA, HellaS, WinoG, ARC-e, ARC-c, and OBQA at 50% pruning rate.

Method	Total Budget (Prune-then-Recover)				Acc.
	Train	GPU (GB)	Param	Tokens	Avg.
Baseline	-	-	-	-	65.45
PCA	0 h	0	0 M	157.1 K	32.73
TransKV	0 h	0	0 M	7.0 M	**54.65**
Mat.KV	56 h	88 × 7	470 M	1.2 B	59.71
TransKV + LoRA	0 h + 0.5 h	0 + 8.3 × 7	0 M + 25.7 M	7.0 M + 298.0 K	**60.61**
Mat.KV + LoRA	56 h + 0.5 h	88 × 7 + 8.3 × 7	470 M + 25.7 M	1.2 B + 298.0 K	62.76
TransKV + LoRA	0 h + 1.3 h	0 + 8.3 × 7	0 M + 25.7 M	7.0 M + 1.1 M	**63.00**
Mat.KV + LoRA	56 h + 1.4 h	88 × 7 + 8.3 × 7	470 M + 25.7 M	1.2 B + 1.1 M	65.15
TransKV + LoRA	0 h + 1.7 h	0 + 8.4 × 7	0 M + 241.4 M	7.0 M + 1.1 M	**65.23**

Difference from MatryoshkaKV (Mat.KV). **1. Training-Free vs. Heavy Pre-training**. MatryoshkaKV relies on 'learned' projection matrices requiring 56 GPU hours on 7 × 96GB H20s (reproduced by us) and massive generic data (1.2B). TransKV employs a training-free statistical initialization. **2. Total Budget & Coexistence**. As shown in Table 3, while LoRA enables general performance recovery, TransKV uniquely bridges the initialization gap via low-cost tuning. Crucially, TransKV+LoRA outperforms Mat.KV+LoRA with a lower total budget. Since both final performance converges close to baseline, TransKV stands out as the most efficient alternative where extensive pre-training is infeasible.

A.8. TransKV for Intra-ImageNet-1K sublabels Sensitivity

In the main paper, we conduct a cross-dataset sensitivity analysis for TransKV. Here, in the appendix, we further investigate its sensitivity to different label subsets within the same dataset. We use ImageNet-1K [4] classification with ViT-Base as the target task and model. The 1,000 ImageNet classes are sequentially partitioned into 10 equal subsets (100 classes each): the first subset contains classes 0–99, the second contains classes 100–199, and so on. This yields 10 subclass-specific sub-datasets. We then construct 11 different projection matrices using identical class-balanced sampling strategies: one from a 15% random sample of the full ImageNet training set, and the remaining ten from 15% samples of each corresponding sub-dataset.

Tables 6 and 7 report the pruning performance of TransKV at 30% and 50% compression ratios, respectively. For each table, we evaluate 11 variants (each equipped with one of the 11 projection matrices) on both the full ImageNet validation set and the 10 sub-datasets. The results

Table 4. Ablation study on projection matrices in both query-key pair and value-output pair.

Dataset				Ratio	CIFAR10	CIFAR100	iNaturalist
Baseline				0%	97.68	90.56	73.80
Proj. for W_Q^i	Proj. for W_K^i	Proj. for W_V^i	Proj. for W_O^i				
P_Q^i	$(P_Q^i)^\top$	P_V^i	$(P_V^i)^\top$	10%	<u>97.62</u>	<u>90.08</u>	**62.11**
P_Q^i	$(P_Q^i)^\top$	P_O^i	$(P_O^i)^\top$		97.39	88.55	59.64
P_Q^i	$(P_Q^i)^\top$	$P_V^i(P_V^i)^\top P_O^i$	$(P_O^i)^\top$		97.33	88.20	58.55
$P_Q^i(P_Q^i)^\top P_K^i$	$P_K^\top$	P_V^i	$(P_V^i)^\top$		97.60	90.14	<u>61.91</u>
$P_Q^i(P_Q^i)^\top P_K^i$	$P_K^\top$	P_O^i	$(P_O^i)^\top$		97.38	88.45	59.67
$P_Q^i(P_Q^i)^\top P_K^i$	$P_K^\top$	$P_V^i(P_V^i)^\top P_O^i$	$(P_O^i)^\top$		97.26	88.05	58.28
P_K	$P_K^\top$	P_V^i	$(P_V^i)^\top$		**97.65**	**90.19**	61.88
P_K	$P_K^\top$	P_O^i	$(P_O^i)^\top$		97.37	88.58	59.47
P_K	$P_K^\top$	$P_V^i(P_V^i)^\top P_O^i$	$(P_O^i)^\top$		97.31	88.27	58.65
P_Q^i	$(P_Q^i)^\top$	P_V^i	$(P_V^i)^\top$	20%	97.50	**89.80**	**60.73**
P_Q^i	$(P_Q^i)^\top$	P_O^i	$(P_O^i)^\top$		96.25	85.75	55.05
P_Q^i	$(P_Q^i)^\top$	$P_V^i(P_V^i)^\top P_O^i$	$(P_O^i)^\top$		95.76	84.66	52.67
$P_Q^i(P_Q^i)^\top P_K^i$	$P_K^\top$	P_V^i	$(P_V^i)^\top$		<u>97.51</u>	89.69	60.36
$P_Q^i(P_Q^i)^\top P_K^i$	$P_K^\top$	P_O^i	$(P_O^i)^\top$		96.18	85.52	54.36
$P_Q^i(P_Q^i)^\top P_K^i$	$P_K^\top$	$P_V^i P_{V^T} P_O^i$	$(P_O^i)^\top$		95.58	84.58	52.44
P_K	$P_K^\top$	P_V^i	$(P_V^i)^\top$		**97.52**	<u>89.79</u>	<u>60.56</u>
P_K	$P_K^\top$	P_O^i	$(P_O^i)^\top$		96.28	86.19	55.12
P_K	$P_K^\top$	$P_V^i(P_V^i)^\top P_O^i$	$(P_O^i)^\top$		95.62	85.16	52.77
P_Q^i	$(P_Q^i)^\top$	P_V^i	$(P_V^i)^\top$	30%	**97.25**	**88.72**	**58.91**
P_Q^i	$(P_Q^i)^\top$	P_O^i	$(P_O^i)^\top$		93.06	76.93	47.92
P_Q^i	$(P_Q^i)^\top$	$P_V^i(P_V^i)^\top P_O^i$	$(P_O^i)^\top$		92.55	73.40	41.82
$P_Q^i(P_Q^i)^\top P_K^i$	$P_K^\top$	P_V^i	$(P_V^i)^\top$		<u>97.16</u>	88.67	58.51
$P_Q^i(P_Q^i)^\top P_K^i$	$P_K^\top$	P_O^i	$(P_O^i)^\top$		92.86	76.04	47.36
$P_Q^i(P_Q^i)^\top P_K^i$	$P_K^\top$	$P_V^i(P_V^i)^\top P_O^i$	$(P_O^i)^\top$		92.31	72.65	41.16
P_K	$P_K^\top$	P_V^i	$(P_V^i)^\top$		**97.25**	<u>88.70</u>	<u>58.68</u>
P_K	$P_K^\top$	P_O^i	$(P_O^i)^\top$		93.20	77.96	48.12
P_K	$P_K^\top$	$P_V^i(P_V^i)^\top P_O^i$	$(P_O^i)^\top$		92.62	74.50	42.24
P_Q^i	$(P_Q^i)^\top$	P_V^i	$(P_V^i)^\top$	40%	**96.87**	<u>86.53</u>	**56.44**
P_Q^i	$(P_Q^i)^\top$	P_O^i	$(P_O^i)^\top$		83.15	59.88	36.73
P_Q^i	$(P_Q^i)^\top$	$P_V^i(P_V^i)^\top P_O^i$	$(P_O^i)^\top$		80.03	53.58	27.29
$P_Q^i(P_Q^i)^\top P_K^i$	$P_K^\top$	P_V^i	$(P_V^i)^\top$		96.68	86.22	55.31
$P_Q^i(P_Q^i)^\top P_K^i$	$P_K^\top$	P_O^i	$(P_O^i)^\top$		92.86	76.04	47.36
$P_Q^i(P_Q^i)^\top P_K^i$	$P_K^\top$	$P_V^i(P_V^i)^\top P_O^i$	$(P_O^i)^\top$		92.31	72.65	41.16
P_K	$P_K^\top$	P_V^i	$(P_V^i)^\top$		<u>96.86</u>	**86.66**	<u>56.11</u>
P_K	$P_K^\top$	P_O^i	$(P_O^i)^\top$		84.50	62.34	37.26
P_K	$P_K^\top$	$P_V^i(P_V^i)^\top P_O^i$	$(P_O^i)^\top$		81.27	56.37	28.02
P_Q^i	$(P_Q^i)^\top$	P_V^i	$(P_V^i)^\top$	50%	**96.08**	**82.90**	<u>49.77</u>
P_Q^i	$(P_Q^i)^\top$	P_O^i	$(P_O^i)^\top$		59.81	24.80	18.84
P_Q^i	$(P_Q^i)^\top$	$P_V^i(P_V^i)^\top P_O^i$	$(P_O^i)^\top$		50.09	16.96	8.78
$P_Q^i(P_Q^i)^\top P_K^i$	$P_K^\top$	P_V^i	$(P_V^i)^\top$		95.60	81.89	48.58
$P_Q^i(P_Q^i)^\top P_K^i$	$P_K^\top$	P_O^i	$(P_O^i)^\top$		62.85	21.98	18.22
$P_Q^i(P_Q^i)^\top P_K^i$	$P_K^\top$	$P_V^i(P_V^i)^\top P_O^i$	$(P_O^i)^\top$		53.11	19.07	10.26
P_K	$P_K^\top$	P_V^i	$(P_V^i)^\top$		<u>95.85</u>	<u>82.77</u>	**50.92**
P_K	$P_K^\top$	P_O^i	$(P_O^i)^\top$		62.24	27.75	20.17
P_K	$P_K^\top$	$P_V^i(P_V^i)^\top P_O^i$	$(P_O^i)^\top$		51.70	19.67	10.03

Table 5. Comparative performance across different pruning rates on Common Voice 17. The traditional pruning methods are applied exclusively to the linear layers (Query, Key, Value, and Output) within the Attention module. On English, TransKV computes its rotation matrices using 5% of the training data; on Cantonese, 100% of the training data is used for the same purpose.

	Ratio	Encoder				Decoder				Encoder-Decoder			
		English		Cantonese		English		Cantonese		English		Cantonese	
		WER	CER	WER	CER	WER	CER	WER	CER	WER	CER	WER	CER
Baseline	0%	21.43	9.54	8.56	1.27	21.43	9.54	8.56	1.27	21.43	9.54	8.56	1.27
L1	10%	181.38	171.73	99.25	344.27	96.78	97.61	1028.05	439.77	99.63	99.71	103.23	98.66
L2		129.63	115.03	95.19	166.15	97.62	98.06	2556.74	1105.09	102.84	102.71	3992.6	1373.19
Random		167.34	145.18	99.59	339.87	123.29	111.8	99.14	110.54	101.53	97.53	100.0	206.05
Taylor		23.7	11.06	14.87	2.52	28.87	14.77	46.94	9.21	30.69	15.97	47.09	9.36
CLOVER		24.24	11.41	14.76	2.58	86.7	74.57	43.11	43.57	89.06	74.32	48.33	34.81
TransKV		**21.8**	**9.77**	**9.61**	**1.42**	**21.63**	**9.73**	**10.29**	**1.53**	**21.89**	**9.89**	**10.44**	**1.56**
L1	20%	99.75	89.48	199.85	392.83	101.94	97.86	388.62	180.91	99.57	97.72	772.47	362.4
L2		106.89	99.15	413.14	668.12	537.25	283.5	19075.07	3939.98	2321.04	1566.55	10549.98	4071.31
Random		123.01	113.62	100.0	1547.37	99.31	99.6	100.0	103.84	99.87	101.5	100.0	160.49
Taylor		33.2	17.75	36.31	8.51	340.63	227.48	1681.86	750.38	86.57	72.32	979.27	424.38
CLOVER		28.48	14.57	33.16	7.68	173.68	156.04	126.89	126.68	229.95	203.35	129.22	118.26
TransKV		**22.29**	**10.09**	**10.1**	**1.66**	**22.05**	**9.98**	**13.11**	**1.97**	**22.54**	**10.26**	**15.06**	**2.38**
L1	30%	100.65	90.45	4102.93	2432.81	100.0	196.65	100.0	2200.72	99.92	95.3	100.0	2200.72
L2		101.28	85.66	825.8	472.47	99.99	98.01	100.0	100.81	98.62	96.46	100.0	100.0
Random		918.9	397.88	130.49	194.18	100.0	98.33	110.4	198.16	2302.09	779.17	21846.6	4336.71
Taylor		83.45	61.61	90.88	75.46	119.88	103.79	3009.84	1039.11	99.69	98.04	712.24	273.45
CLOVER		60.95	45.53	72.4	35.37	122.01	104.35	200.53	132.9	156.97	132.02	196.24	128.92
TransKV		**23.41**	**10.87**	**13.67**	**2.66**	**24.41**	**11.6**	**21.55**	**3.43**	**25.8**	**12.82**	**28.09**	**4.96**
L1	40%	117.34	100.27	14704.39	8735.62	100.0	382.85	100.0	2200.72	100.0	410.92	100.0	2200.72
L2		132.9	111.35	904.54	523.08	99.99	102.31	100.0	2200.72	100.0	386.89	100.0	2200.72
Random		354.26	1132.77	132.22	2077.18	100.42	298.52	100.0	103.59	100.0	100.0	100.0	100.0
Taylor		157.73	158.86	160.08	448.78	96.89	97.16	998.95	447.64	100.05	410.93	100.0	702.2
CLOVER		149.68	130.99	90.54	104.54	93.42	93.59	100.0	100.0	107.28	101.28	102.48	99.98
TransKV		**26.36**	**13.09**	**20.84**	**3.87**	**35.23**	**19.39**	**47.99**	**15.06**	**35.47**	**18.84**	**56.93**	**17.82**
L1	50%	920.59	1114.77	559.37	1519.47	99.99	98.37	**100.0**	**100.0**	100.0	98.67	**100.0**	2200.72
L2		582.19	535.54	448.59	1259.23	99.97	98.5	**100.0**	2117.88	96.11	95.43	9539.5	5018.09
Random		99.97	562.43	341.38	3725.61	100.0	410.9	**100.0**	986.21	652.36	272.75	**100.0**	3669.87
Taylor		1227.33	1194.97	100.08	545.07	98.42	168.64	**100.0**	100.01	99.98	98.34	**100.0**	**100.0**
CLOVER		205.37	180.69	99.96	436.67	**96.86**	96.58	**100.0**	**100.0**	99.76	101.28	**100.0**	**100.0**
TransKV		**38.49**	**23.22**	**36.99**	**9.08**	106.44	**82.12**	100.3	178.14	**85.5**	**60.6**	101.92	139.25

clearly demonstrate that TransKV achieves the highest performance when the projection matrix is derived from the same data distribution as the evaluation set. This consistent matching effect strongly confirms TransKV's robust data-driven capability for identifying intra-dataset redundancies.

A.9. TransKV for SmolLM3-3B on MMLU Benchmark

Here, we further evaluate TransKV on SmolLM3-3B [3], a state-of-the-art 3-billion-parameter dense language model that incorporates both RoPE and non-RoPE operations. Performance is measured on the Massive Multitask Language Understanding (MMLU) benchmark [6], which comprises multiple-choice questions spanning 57 diverse subjects and serves as a standard evaluation suite for large language models. We compare TransKV against several baselines (L1, L1, L2, Random, Taylor, and CLOVER) under two settings: (1) Non-RoPE pruning only, and (2) joint RoPE + Non-RoPE pruning, across a range of pruning ratios.

As shown in Table 8, in the Non-RoPE pruning setting, TransKV consistently outperforms L1, L2, Random, and Taylor across all pruning ratios in both individual subtasks and overall MMLU score. TransKV achieves performance comparable to CLOVER in this setting, which is expected since Non-RoPE pruning affects only the final 9 layers of SmolLM3-3B, leaving relatively limited redundancy for compression. When RoPE and Non-RoPE components are pruned jointly, TransKV substantially surpasses all competing methods, whereas CLOVER is inapplicable due to its lack of RoPE support. These results highlight the broader applicability and superior effectiveness of TransKV when handling diverse attention mechanisms and positional encoding schemes in modern language models.

References

[1] Yongqi An, Xu Zhao, Tao Yu, Ming Tang, and Jinqiao Wang. Fluctuation-based adaptive structured pruning for large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pages 10865–10873, 2024. 1, 4

[2] Saleh Ashkboos, Maximilian L Croci, Marcelo Gennari do Nascimento, Torsten Hoefler, and James Hensman. Slicegpt: Compress large language models by deleting rows and columns. In *The Twelfth International Conference on Learning Representations*, 2024. 4

[3] Elie Bakouch, Loubna Ben Allal, Anton Lozhkov, Nouamane Tazi, Lewis Tunstall, Carlos Miguel Patiño, Edward Beeching, Aymeric Roucher, Aksel Joonas Reedi, Quentin Gallouédec, Kashif Rasul, Nathan Habib, Clémentine Fourrier, Hynek Kydlicek, Guilherme Penedo, Hugo Larcher, Mathieu Morlon, Vaibhav Srivastav, Joshua Lochner, Xuan-Son Nguyen, Colin Raffel, Leandro von Werra, and Thomas Wolf. SmolLM3: smol, multilingual, long-context reasoner. https://huggingface.co/blog/smollm3, 2025. 8

[4] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009. 5

[5] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020. 1

[6] Dan Hendrycks, Collin Burns, Steven Basart, Andy Zou, Mantas Mazeika, Dawn Song, and Jacob Steinhardt. Measuring massive multitask language understanding. In *International Conference on Learning Representations*, 2021. 8

[7] Mary Ann Marcinkiewicz. Building a large annotated corpus of english: The penn treebank. *Using Large Corpora*, 273: 31, 1994. 1

[8] Fanxu Meng, Pingzhi Tang, Fan Jiang, and Muhan Zhang. CLOVER: Cross-layer orthogonal vectors pruning. In *International Conference on Machine Learning*, 2025. 1

[9] Stephen Merity, Caiming Xiong, James Bradbury, and Richard Socher. Pointer sentinel mixture models. In *International Conference on Learning Representations*, 2017. 1

[10] Qwen, :, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chengyuan Li, Dayiheng Liu, Fei Huang, Haoran Wei, Huan Lin, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiaxi Yang, Jingren Zhou, Junyang Lin, Kai Dang, Keming Lu, Keqin Bao, Kexin Yang, Le Yu, Mei Li, Mingfeng Xue, Pei Zhang, Qin Zhu, Rui Men, Runji Lin, Tianhao Li, Tianyi Tang, Tingyu Xia, Xingzhang Ren, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yu Wan, Yuqiong Liu, Zeyu Cui, Zhenru Zhang, and Zihan Qiu. Qwen2.5 technical report, 2025. 1

Table 6. TransKV results on ImageNet with a 30% pruning ratio on ViT-Base (accuracy). The global projection for ImageNet is constructed using 15% of training data. For sub-tasks (e.g., 000–099), the projection matrices are derived from subsets of this 15% training data, restricted to samples belonging to the corresponding class labels of each sub-task. The box highlight the best result.

Projection \ Dataset	Whole	000-099	100-199	200-299	300-399	400-499	500-599	600-699	700-799	800-899	900-999
Proj. from whole	67.93	73.26	70.94	68.88	76.06	63.26	64.86	63.16	66.46	62.18	70.28
Proj. from 000-099	66.21	76.08	70.34	67.94	76.18	60.10	61.36	59.10	63.14	58.46	69.44
Proj. from 100-199	67.19	73.46	75.24	68.70	75.36	60.66	62.66	60.92	64.26	60.54	70.14
Proj. from 200-299	66.88	72.64	69.94	71.38	75.22	60.96	62.64	61.30	64.86	60.20	69.64
Proj. from 300-399	66.59	73.60	70.84	68.56	78.66	60.12	61.76	59.82	63.80	59.72	68.98
Proj. from 400-499	66.28	68.24	66.56	63.78	72.30	68.48	65.04	62.92	65.16	62.00	68.32
Proj. from 500-599	66.41	68.26	66.44	63.54	72.24	63.50	70.46	62.50	66.32	62.44	68.36
Proj. from 600-699	66.47	68.24	67.46	64.58	73.04	62.92	65.18	68.12	65.46	62.12	67.54
Proj. from 700-799	66.24	68.52	66.76	63.80	71.48	62.98	65.30	62.38	70.92	62.38	67.84
Proj. from 800-899	66.55	68.76	67.08	64.58	72.42	63.26	65.30	63.12	65.24	67.22	68.54
Proj. from 900-999	66.11	71.14	67.44	64.18	73.54	61.18	63.10	60.48	63.54	61.24	75.26

Table 7. TransKV results on ImageNet with a 50% pruning ratio on ViT-Base (accuracy). The global projection for ImageNet is constructed using 15% of training data. For sub-label dataset (e.g., 000–099), the projection matrices are derived from subsets of this 15% training data, restricted to samples belonging to the corresponding class labels of each sub-task. The box highlight the best result.

Projection \ Dataset	Whole	000-099	100-199	200-299	300-399	400-499	500-599	600-699	700-799	800-899	900-999
Proj. from whole	67.93	73.26	70.94	68.88	76.06	63.26	64.86	63.16	66.46	62.18	70.28
Proj. from 000-099	66.21	76.08	70.34	67.94	76.18	60.10	61.36	59.10	63.14	58.46	69.44
Proj. from 100-199	67.19	73.46	75.24	68.70	75.36	60.66	62.66	60.92	64.26	60.54	70.14
Proj. from 200-299	66.88	72.64	69.94	71.38	75.22	60.96	62.64	61.30	64.86	60.20	69.64
Proj. from 300-399	66.59	73.60	70.84	68.56	78.66	60.12	61.76	59.82	63.80	59.72	68.98
Proj. from 400-499	66.28	68.24	66.56	63.78	72.30	68.48	65.04	62.92	65.16	62.00	68.32
Proj. from 500-599	66.41	68.26	66.44	63.54	72.24	63.50	70.46	62.50	66.32	62.44	68.36
Proj. from 600-699	66.47	68.24	67.46	64.58	73.04	62.92	65.18	68.12	65.46	62.12	67.54
Proj. from 700-799	66.24	68.52	66.76	63.80	71.48	62.98	65.30	62.38	70.92	62.38	67.84
Proj. from 800-899	66.55	68.76	67.08	64.58	72.42	63.26	65.30	63.12	65.24	67.22	68.54
Proj. from 900-999	66.11	71.14	67.44	64.18	73.54	61.18	63.10	60.48	63.54	61.24	75.26

Table 8. Zero-shot accuracy results on SmolLM3-3B in MMLU benchmark (57 sub-tasks). Attention in SmolLM3-3B is GQA. It has 36 attention layers, where each three RoPE GQA layer follows by a None-RoPE GQA layer. In the table, Soc. Sci., Hum. and Oth. denote Social Sciences, Humanities, and Other, respectively. Note that, the average metric is calculated by each sub-task accuracy.

		Non-RoPE (9 Layers)					RoPE and Non-RoPE (36 Layers)				
	Ratio	STEM	Soc. Sci.	Hum.	Oth.	Avg.	STEM	Soc. Sci.	Hum.	Oth.	Avg.
Baseline	0%	0.55	0.68	0.66	0.61	0.61	0.55	0.68	0.66	0.61	0.61
L1	10%	0.41	0.52	0.54	0.50	0.49	0.23	0.25	0.25	0.24	0.24
L2		0.43	0.52	0.53	0.50	0.49	0.23	0.26	0.24	0.24	0.24
Random		0.52	0.64	0.65	0.60	0.60	0.35	0.44	0.42	0.42	0.40
Taylor		0.53	0.65	0.64	0.59	0.59	0.40	0.51	0.50	0.50	0.46
CLOVER		**0.54**	0.66	0.65	0.60	0.60	unsupported				
TransKV		**0.54**	**0.68**	**0.66**	**0.62**	**0.61**	**0.48**	**0.60**	**0.57**	**0.56**	**0.54**
L1	20%	0.40	0.50	0.53	0.49	0.48	0.25	0.25	0.25	0.24	0.25
L2		0.41	0.50	0.53	0.47	0.47	0.24	0.28	0.25	0.25	0.25
Random		0.49	0.62	0.60	0.57	0.56	0.23	0.23	0.24	0.25	0.24
Taylor		0.50	0.61	0.60	0.57	0.56	0.27	0.28	0.24	0.26	0.26
CLOVER		0.54	0.66	0.64	**0.62**	**0.61**	unsupported				
TransKV		**0.55**	**0.67**	**0.65**	0.61	**0.61**	**0.45**	**0.57**	**0.53**	**0.53**	**0.51**
L1	30%	0.40	0.53	0.53	0.50	0.48	0.26	0.31	0.25	0.25	0.26
L2		0.41	0.51	0.52	0.48	0.47	0.24	0.22	0.24	0.26	0.24
Random		0.46	0.57	0.56	0.53	0.52	0.22	0.21	0.24	0.24	0.23
Taylor		0.47	0.58	0.56	0.56	0.54	0.22	0.21	0.24	0.24	0.23
CLOVER		0.53	0.66	**0.64**	**0.61**	**0.60**	unsupported				
TransKV		**0.55**	**0.67**	**0.64**	**0.61**	**0.60**	**0.43**	**0.52**	**0.49**	**0.47**	**0.47**
L1	40%	0.42	0.54	0.53	0.51	0.49	0.22	0.21	0.24	0.24	0.23
L2		0.42	0.54	0.53	0.50	0.49	0.22	0.21	0.24	0.25	0.23
Random		0.43	0.53	0.52	0.48	0.48	0.22	0.21	0.24	0.24	0.23
Taylor		0.45	0.56	0.53	0.54	0.51	0.22	0.21	0.24	0.24	0.23
CLOVER		0.51	0.63	0.62	0.58	0.57	unsupported				
TransKV		**0.54**	**0.65**	**0.64**	**0.59**	**0.60**	**0.39**	**0.46**	**0.44**	**0.45**	**0.43**
L1	50%	0.43	0.55	0.53	0.50	0.49	0.22	0.21	0.24	0.24	0.23
L2		0.42	0.54	0.52	0.50	0.49	0.22	0.21	0.24	0.24	0.23
Random		0.43	0.53	0.51	0.48	0.48	0.28	0.33	0.24	0.26	0.27
Taylor		0.45	0.58	0.54	0.54	0.52	0.28	0.29	0.25	0.24	0.26
CLOVER		0.52	0.61	0.61	0.57	0.57	unsupported				
TransKV		**0.52**	**0.65**	**0.63**	**0.58**	**0.58**	**0.35**	**0.43**	**0.38**	**0.40**	**0.38**